# Python: Recap IV

This notebook was generated in **solutions** mode.

---

## Exercise 1: ASCII sum substring

### Exercise Description

You are given a number `N` and a string `S`. In Python, a character can be converted to a numeric code (its ASCII representation) using the function `ord()`. For example, `ord('A')` is 65, `ord('B')` is 66, `ord('a')` is 97, `ord('1')` is 49, and so on.

Write a function `ascii_sum(N, S)` that returns `True` if `S` contains a substring whose **sum of ASCII codes** is exactly `N`, and `False` otherwise.

Example: if `N = 180` and `S = 'CAB19'`, the function must return `True` because there exists the substring `'AB1'` such that `ord('A') + ord('B') + ord('1') = 65 + 66 + 49 = 180`.

### Your solution

```python
def ascii_sum(N, S):
    for i in range(len(S)):
        for j in range(i + 1, len(S) + 1):
            this_substring = S[i:j]
            this_sum = sum(ord(letter) for letter in this_substring)
            if this_sum == N:
                return True
    return False
```

Test your solution

```python
# Test case 1 (Example)
N = 180
S = 'CAB19'
assert ascii_sum(N, S) is True

# Test case 2
N = 180
S = 'CAB91'
assert ascii_sum(N, S) is False

# Test case 3
N = 200
S = 'Hello, World!'
assert ascii_sum(N, S) is False

# Test case 4
N = 65  # ASCII code for 'A'
S = 'ABCDEF'
assert ascii_sum(N, S) is True
```

### Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 2: Flip if upper

### Exercise Description

Write a function `my_function(s)` that receives a string `s` (a sentence where words are separated by whitespaces) and returns a new string built according to the following rule: if a word in `s` begins with an uppercase letter, then the word must be **reversed** in the new string; otherwise it must be left intact.

The order of the words between the input and the output string must not change. If the input string is empty, the function must return `None`.

Note: the examples include a trailing space at the end of the returned string.

### Your solution

```
def my_function(s):
    # let's check if the input string is empty
    if s == '':
        return None

    # if not, then split the string into words
    lst = s.split()

    # initialize an empty string that we will fill with the (eventually reversed) wor
    s_new = ''
    for word in lst:
        if word[0].isupper():
            s_new += word[::-1] + ' '
        else:
            s_new += word + ' '

    # job done
    return s_new
```

Test your solution

```
# Test case 1 (Example)
s = 'hello World'
t = my_function(s)
assert t == 'hello dlroW '

# Test case 2
s = ''
t = my_function(s)
assert t is None
```

```python
# Test case 3
s = 'Python is AWESOME'
t = my_function(s)
assert t == 'nohtyP is EMOSEWA '

# Test case 4
s = 'viva david parenzo'
t = my_function(s)
assert t == 'viva david parenzo '
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 3: Longest word in string

Exercise Description

Write a function `my_function(s)` that receives a string `s` (representing a sentence where words are separated by whitespaces) and returns the **longest word** in `s`.

If the input string is empty, the function must return None. If there are more than one longest words, the function must return the **first** one.

Your solution

```
def my_function(s):
    # let's check if the input string is empty
    if s == '':
        return None

    # if not, then split the string into words
    lst = s.split()

    # initialize variables to keep track of the longest word
    max_length = 0

    # iterate over the words
    for word in lst:
        # do we have a new longest word?
        if len(word) > max_length:
            max_length = len(word)
            longest_word = word

    return longest_word
```

Test your solution

```
# Test case 1 (Example)
s = 'This is an example sentence with some long words.'
assert my_function(s) == 'sentence'
```

```python
# Test case 2
s = 'The quick brown fox jumps over the lazy dog.'
assert my_function(s) == 'quick'

# Test case 3
s = ''
assert my_function(s) is None

# Test case 4
s = 'Ragazzi ma se po senti Parento su lasette che fa il grande storico'
assert my_function(s) == 'Ragazzi'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 4: Periodic String Group

Exercise Description

Write a function `my_function(X, Y, S)` that, given two integer values `X` and `Y` and a string `S` of length 1 (a character), returns a string with `Y` **groups** of characters. Each group consists of `X` copies of `S`. Groups are separated by `'-'`.

You can assume that `X` and `Y` are greater than or equal to 0. If `X == 0`, groups are empty and no `'-'` separators should be used (the function should return the empty string).

Hint: you can use the `*` operator to obtain sequences of equal characters, e.g., `'x' * 7 == 'xxxxxxx'`.

Your solution

```
def my_function(X, Y, S):
    new_s = ''
    if X == 0:
        return new_s
    for i in range(Y - 1):
        new_s = new_s + (X * S) + '-'
    new_s = new_s + (X * S)
    return new_s
```

Test your solution

```
# Test case 1 (Example)
s1 = my_function(4, 2, 'A')
assert s1 == 'AAAA-AAAA'

# Test case 2
s1 = my_function(1, 10, 'A')
```

```
assert s1 == 'A-A-A-A-A-A-A-A-A-A-A'

# Test case 3
s1 = my_function(0, 5, 'A')
assert s1 == ''

# Test case 4
s1 = my_function(3, 5, 'A')
assert s1 == 'AAA-AAA-AAA-AAA-AAA'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 5: Periodic String

Exercise Description

Write a function `my_function(X, Y, S)` that, given two integer values `X` and `Y` and a string `S` of length 1 (a character), returns a string with `X` characters `S` separated by `Y` characters `'-'`. You can assume that `X` and `Y` are greater than or equal to 0.

If `X == 1` no `'-'` characters should be added, independently of the value of `Y`.

Hint: you can use the `*` operator to obtain sequences of equal characters (e.g., `'x' * 7 == 'xxxxxxx'`).

Your solution

```
def my_function(X, Y, S):
    new_s = ''
    if X != 0:
        for i in range(X - 1):
            new_s = new_s + S + (Y * '-')
        new_s = new_s + S
    return new_s
```

Test your solution

```
# Test case 1 (Example)
s1 = my_function(3, 2, 'A')
assert s1 == 'A--A--A'

# Test case 2
s1 = my_function(1, 10, 'A')
assert s1 == 'A'

# Test case 3
s1 = my_function(0, 5, 'A')
assert s1 == ''
```

```python
# Test case 4
s1 = my_function(4, 5, 'A')
assert s1 == 'A-----A-----A-----A'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 6: Remove Char

Exercise Description

Write a function `my_function(S, C)` that, given as parameters a string `S` and another string `C` of length 1 (a character), returns a string `T` that is equal to `S` except that it contains **no occurrence** of character `C`.

Your solution

```python
def my_function(S, C):
    new_s = ''
```

```python
    for letter in S:
        if letter != C:
            new_s = new_s + letter
    return new_s
```

Test your solution

```python
# Test case 1 (Example)
s = 'I study at Luiss'
c = 's'
t = my_function(s, c)
assert t == 'I tudy at Lui'

# Test case 2
s = 'xxxxxxo'
c = 'x'
t = my_function(s, c)
assert t == 'o'

# Test case 3
s = 'oxxxxxxx 000'
c = 'x'
t = my_function(s, c)
assert t == 'o 000'

# Test case 4
s = 'oxxxxxxxo'
c = 's'
t = my_function(s, c)
assert t == 'oxxxxxxxo'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 7: Replace Substring

Exercise Description

Write a function `my_function(S, R, C)` that, given as parameters a string `S`, a string `R` and another string `C` of length `1` (a character), returns a string `T` that is equal to `S` except that each occurrence of character `C` is **replaced** by string `R`.

Your solution

```
def my_function(S, R, C):
    new_s = ''
    for letter in S:
        if letter == C:
            new_s = new_s + R
        else:
```

```
            new_s = new_s + letter
    return new_s
```

Test your solution

```
# Test case 1 (Example)
s = 'I study at Luiss'
r = 'REPLACE'
c = 's'
t = my_function(s, r, c)
assert t == 'I REPLACEtudy at LuiREPLACEREPLACE'

# Test case 2
s = 'I REPLACEddd'
r = 'REPLACE'
c = 'D'
t = my_function(s, r, c)
assert t == 'I REPLACEddd'

# Test case 3
s = 'D'
r = 'REPLACE'
c = 'D'
t = my_function(s, r, c)
assert t == 'REPLACE'

# Test case 4
s = ''
r = 'REPLACE'
c = 'D'
```

```
t = my_function(s, r, c)
assert t == ''
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 8: Driving License

Exercise Description

Write a function `my_function(license_plate)` that validates a driving license plate format.

The function should return `True` if the license plate follows the correct format, and `False` otherwise.

A valid license plate must:

- Be exactly 7 characters long
- Have exactly 4 letters followed by exactly 3 digits
- All letters must be uppercase

For example: "ABCD123" is valid, but "ABC123", "ABCDE123", or "ABCD12" are not valid.

Your solution

```
def my_function(license_plate):
    if len(license_plate) != 7:
        return False
    if not license_plate[:4].isalpha() or not license_plate[:4].isupper():
        return False
    if not license_plate[4:].isdigit():
        return False
    return True
```

Test your solution

```
# Test case 1 (Example)
result = my_function("ABCD123")
assert result == True

# Test case 2
result = my_function("ABC123")
assert result == False

# Test case 3
result = my_function("ABCDE123")
```

```python
assert result == False

# Test case 4
result = my_function("ABCD12")
assert result == False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 9: Reverse Order

Exercise Description

Write a Python function `my_function(s)` that takes a string `s` as an argument and returns a new **list of words** that contains the words in `s` in reverse order.

For example, if `s = 'I love you'`, then `my_function(s)` should return `['you', 'love', 'I']`.

Hint: you can use the `split()` method to split the string into a list of words.

Your solution

```python
def my_function(s):
    s = s.split()
    return s[::-1]
```

Test your solution

```python
# Test case 1 (Example)
s1 = 'I love you'
assert my_function(s1) == ['you', 'love', 'I']

# Test case 2
s2 = 'The quick brown fox'
assert my_function(s2) == ['fox', 'brown', 'quick', 'The']

# Test case 3
s3 = 'Hello World'
assert my_function(s3) == ['World', 'Hello']

# Test case 4
s4 = 'Python is fun'
assert my_function(s4) == ['fun', 'is', 'Python']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 10: Alien race

Exercise Description

It's 2027. Earth has been invaded by three alien races: the Greys, the Reptilians, and the Annunaki. A running race is organized to decide which race will dominate. At the end of the race, a list is provided with the order in which the participants finished. Each element of the list is in the format `participant-race`.

The score of a race is determined by **adding the positions** of all its participants, and the race with the **lowest score** wins.

Write a function `my_function(order_of_arrival, race)` that calculates the score obtained by a specific race. Positions are 1-based (the first element has position 1). If the list does not include that race among the participants, the result should be `0`.

Your solution

```
def my_function(order_of_arrival, race):
    score = 0
    i = 0
    for el in order_of_arrival:
```

```
        i += 1
        r = el.split('-')[1]
        if r == race:
            score += i
    return score
```

Test your solution

```
# Test case 1
order_of_arrival = ['Gix-Annunaki', 'Tom-Grey', 'XYZ-Reptilian', 'Alex-Annunaki', 'Ar
race = 'Annunaki'
assert my_function(order_of_arrival, race) == 22

# Test case 2
race = 'Grey'
assert my_function(order_of_arrival, race) == 26

# Test case 3
order_of_arrival = ['Gix-Annunaki', 'Tom-Grey', 'Alex-Annunaki', 'AngelFab-Grey', 'Mi
race = 'Reptilian'
assert my_function(order_of_arrival, race) == 0

# Test case 4
order_of_arrival = ['Gix-Annunaki', 'Tom-Grey', 'XYZ-Reptilian', 'Alex-Annunaki', 'Ar
race = 'Humans'
assert my_function(order_of_arrival, race) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 11: Remove numeric duplicates and sort

### Exercise Description

Write a function `my_function(L)` that, given a list of numbers `L`, returns a new list containing the same numbers **without duplicates**.

The numbers in the output list must be sorted from the smallest to the largest. If the input list is empty, the function should return an empty list.

### Your solution

```
def my_function(L):
    L_new = []
    for item in L:
        if item not in L_new:
            L_new.append(item)
```

```
    L_new = sorted(L_new)
    return L_new
```

Test your solution

```
# Test case 1 (Example)
assert my_function([10, 2, 3, 5, 2, 10, 3]) == [2, 3, 5, 10]

# Test case 2
assert my_function([]) == []

# Test case 3
assert my_function([1, 1, 1, 1, 1, 1]) == [1]

# Test case 4
assert my_function([1, -2, 3, -4, 5, -6, 7, -8, 9, -10] * 1000) == [-10, -8, -6, -4,
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 12: Euro To Dollars

## Exercise Description

Write a function `my_function(L)` that is given a list `L` of numbers representing coins in euro. The function should return the **sum of the coins converted to dollars**, where 1 euro corresponds to 1.22 dollars.

If there are no items in the list, the function should return `0.0` (a float).

## Your solution

```
def my_function(L):
    s = 0
    for n in L:
        s = s + n * 1.22
    return s
    # return sum([item * 1.22 for item in L])
```

## Test your solution

```
# Test case 1 (Example)
assert my_function([3, 2, 4]) == 10.98

# Test case 2
assert my_function([0, 0, 1]) == 1.22

# Test case 3
```

```python
assert my_function([2, 5, 12]) == 23.18

# Test case 4
assert float(my_function([])) == 0.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 13: H-index

Exercise Description

The *h*-index is a common index to quantify the scientific impact of a researcher. For a given researcher, you are given a list `papers` of `n` integer numbers, where `n` is the number of papers. In position *i* you have the number of citations of the *i*-th paper.

The *h*-index is defined as the largest number `h` such that there are **at least `h` papers with at least `h` citations**.

Write a function `my_function(papers)` that returns the h-index of the researcher. You can assume the list contains only integers. If the input list is empty, the function must return `None`.

Your solution

```
def my_function(citations):
    n = len(citations)
    if n == 0:
        return None
    citations_srt = sorted(citations, reverse=True)
    h = 0
    for i in range(n):
        if citations_srt[i] >= i + 1:
            h += 1
    return h
```

Test your solution

```python
# Test case 1 (Example)
assert my_function([24, 2, 36, 2, 3]) == 3

# Test case 2
assert my_function([3, 0, 6, 1, 5]) == 3

# Test case 3
assert my_function([]) is None

# Test case 4
assert my_function([24, 6, 12, 36, 2, 2]) == 4
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 14: Km To Miles

Exercise Description

Write a function `my_function(L)` that is given a list `L` of numbers representing distances in kilometers. The function should return the **sum of the distances converted to miles**, where 1 kilometer corresponds to 0.62 miles.

If there are no items in the list, the function should return `0`.

Your solution

```
def my_function(L):
    return sum([item * 0.62 for item in L])
```

Test your solution

```
# Test case 1 (Example)
assert my_function([3, 2, 4]) == 5.58

# Test case 2
assert my_function([3]) == 1.8599999999999999

# Test case 3
assert float(my_function([0])) == 0.0

# Test case 4
assert float(my_function([])) == 0.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 15: Name Ends With

Exercise Description

Write a function `my_function(L1, L2)` that takes two lists as parameters:

- each element in `L1` is the name of a person,
- each element in `L2` is a character (a string of length 1).

The function shall return a list `L3` containing the persons whose name **ends with** a letter in `L2`. Elements in `L3` should appear in the same order in which they appear in `L1`.

If `L1` is empty or does not contain any person whose name ends with a letter in `L2`, then `L3` must be an empty list.

Your solution

```python
def my_function(L1, L2):
    L3 = []
    for name in L1:
        for letter in L2:
            if name.endswith(letter):
                L3.append(name)
                break
    return L3
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(["Giorgio", "Irene", "Antonio", "Carla"], ["o", "e"]) == ["Giorgio

# Test case 2
assert my_function(["Giorgio", "Irene", "Antonio", "Carla"], ["e", "x"]) == ["Irene"]

# Test case 3
assert my_function(["Giorgio", "Irene", "Antonio", "Carlx"], ["e", "x"]) == ["Irene",

# Test case 4
assert my_function(["Giorgio", "Irene", "Antonio", "Carlx"], ["v", "z"]) == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

[🔗 Visualize YOUR solution in Python Tutor]

---

# Exercise 16: Nevio the Ironed

Exercise Description

"Nevio the Ironed" is a fervent gambler who has accumulated a series of debts with banks. He is ordered to repay the amounts owed. Nevio currently has in his pocket a `winning_sum` of money he just won by betting on a horse.

You are given a list `debts` where each element has the format `'amount-repaid'`:

- `amount` is an integer amount of money,
- `repaid` is `0` if the amount has **not** yet been settled and `1` if it has already been repaid.

Write a function `my_function(debts, winning_sum)` that returns `True` if Nevio can settle **all unpaid debts** using `winning_sum`, and `False` otherwise.

Your solution

```python
def my_function(debts, winning_sum):
    for debt in debts:
        if debt[-1] == '0':
```

```python
            winning_sum -= int(debt[:-2])
    return winning_sum >= 0
```

Test your solution

```python
# Test case 1
debts = ['300-1', '200-0', '100-0']
winning_sum = 500
assert my_function(debts, winning_sum) is True

# Test case 2
debts = ['500-0', '200-0', '100-1']
winning_sum = 500
assert my_function(debts, winning_sum) is False

# Test case 3
debts = ['500-0', '200-0', '100-1']
winning_sum = 400
assert my_function(debts, winning_sum) is False

# Test case 4
debts = ['500-0', '200-0', '100-1']
winning_sum = 300
assert my_function(debts, winning_sum) is False
```

### Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 17: No escape for Earth

### Exercise Description

Some theories suggest the existence of a planet called Nibiru or Planet X. Scientists have identified a series of meteors approaching Earth. For each meteor, they recorded its radius and whether its trajectory is directed towards Earth.

Data are given in a list `data`, where each element has the format `'radius-trajectory'`:

- `radius` is a number,
- `trajectory` is `0` if it is **not** directed towards Earth, and `1` otherwise.

You are also given:

- `circumference`: the **maximum acceptable circumference** for a meteor,
- `limit`: the **maximum number of meteors** directed towards Earth that we can withstand.

Write a function `my_function(data, circumference, limit)` that returns:

- `False` if **any** meteor has circumference greater than `circumference`, or if the number of meteors with trajectory `1` is greater than `limit`,
- `True` otherwise.

You can use `math.pi` and the formula `circ = radius * radius * pi`.

Your solution

```
import math

def my_function(data, circumference, limit):
    count = 0
    for i in data:
        i = i.split('-')
        if float(i[0]) * float(i[0]) * math.pi > circumference:
            return False
        if i[1] == '1':
            count += 1
    if count > limit:
        return False
    else:
        return True
```

Test your solution

```
# Test case 1
data = ['100-1', '200-1', '300-0', '400-1', '500-0']
circumference = 400
limit = 2
assert my_function(data, circumference, limit) is False

# Test case 2
limit = 1
assert my_function(data, circumference, limit) is False

# Test case 3
limit = 3
assert my_function(data, circumference, limit) is False

# Test case 4
circumference = 3000000
limit = 4
assert my_function(data, circumference, limit) is True
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 [Visualize YOUR solution in Python Tutor]

---

# Exercise 18: Person and first letter

Exercise Description

Write a function `my_function(L1, L2)` that takes two lists as parameters:

- each element in list `L1` is the name of a person,
- each element in list `L2` is a character (a string of length `1`).

The function shall return a list `L3` containing the persons whose name **begins with** a letter ir `L2`. Elements in `L3` should appear in the same order in which they appear in `L1`.

If `L1` is empty or does not contain persons whose name begins with a letter in `L2`, then `L3` must be an empty list.

Your solution

```
def my_function(L1, L2):
    L3 = []
    for name in L1:
        initial = name[0]
        if initial in L2:
            L3.append(name)
    return L3
```

Test your solution

```
# Test case 1 (Example)
assert my_function(['Giorgio', 'Irene', 'Antonio', 'Giuseppe'], ['I', 'G']) == ['Gior

# Test case 2
assert my_function(['Giorgio', 'Mino', 'Antonio', 'Marco'], ['I', 'M']) == ['Mino', '

# Test case 3
assert my_function(['Giorgio', 'Mino', 'Antonio', 'Marco'], ['I', 'S']) == []

# Test case 4
assert my_function([], ['I', 'M']) == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 19: Products of Elements

Exercise Description

Write a function `my_function(L1, L2)` that takes as parameters two lists of integer numbers.

The function shall return an empty list if `L1` and `L2` are empty **or** do not have the same length. Otherwise, it shall return a list `L3` containing the products of elements in the same position in `L1` and `L2`.

Your solution

```
def my_function(L1, L2):
    L3 = []
    # do not iterate if lists are empty
    if len(L1) == 0 and len(L2) == 0:
        return L3
    # do not iterate if they have different lengths
    if len(L1) != len(L2):
        return L3
    # otherwise, do
    for i in range(len(L2)):
        L3.append(L2[i] * L1[i])
    return L3
```

Test your solution

```
# Test case 1 (Example)
assert my_function([10, 20, 30], [100, 200, 300]) == [1000, 4000, 9000]
```

```python
# Test case 2
assert my_function([10, 30], [100, 300]) == [1000, 9000]

# Test case 3
assert my_function([10, 30, 15], [100, 300]) == []

# Test case 4
assert my_function([], []) == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 20: Remove duplicates (strings)

Exercise Description

Define a function `my_function(lst)` that takes a list of strings `lst` as input.

The function shall return a list containing the same elements of `lst`, but **without duplicates**: if a string `s` appears more than once in `lst`, only the **first** occurrence of `s` should be copied to the output list.

If the input list `lst` is empty, the function should return an empty list.

Your solution

```
def my_function(lst):
    output = []
    for item in lst:
        if item not in output:
            output.append(item)
    return output
```

Test your solution

```
# Test case 1 (Example)
a = ['a', 'good', 'day', 'is', 'good', 'a', 'bad', 'day', 'is', 'not', 'good']
assert my_function(a) == ['a', 'good', 'day', 'is', 'bad', 'not']

# Test case 2
a = ['test', 'test', 'test', 'test', 'test', 'test', 'test', 'test']
assert my_function(a) == ['test']

# Test case 3
a = ['test', 'test', 'hello', 'test', 'hello', 'test', 'hello', 'test']
assert my_function(a) == ['test', 'hello']

# Test case 4
a = []
assert my_function(a) == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 21: Reptilians

Exercise Description

The Reptilians are among us! An intelligence agency has revealed that many presidents of the world's nations are actually clones controlled by the Reptilians. The clones have specific characteristics:

- They have odd-length names.
- They have light-colored eyes.
- They are over 40 years old.

Each president is represented as a string in the format `'name-eyes-age'`, where `eyes` can be either `'light'` or `'dark'`.

Write a function `my_function(presidents)` that returns the number of presidents that are clones according to the rules above.

Your solution

```
def my_function(presidents):
    number = 0
    for president in presidents:
        p_split = president.split('-')
        name = p_split[0]
        eyes = p_split[1]
        age = p_split[2]
        if len(name) % 2 == 1 and eyes == 'light' and int(age) > 40:
            number += 1
    return number
```

Test your solution

```
# Test cases 1
presidents = ['John-light-45', 'George-dark-54', 'Barack-dark-55', 'Bill-light-68', '
assert my_function(presidents) == 1

# Test cases 2
presidents = ['John-light-45', 'George-dark-54', 'Barack-dark-55', 'Bill-light-68', '
assert my_function(presidents) == 2

# Test cases 3
presidents = ['John-light-40', 'George-dark-54', 'Barack-dark-55', 'Bill-dark-68', 'F
assert my_function(presidents) == 0
```

```python
# Test cases 4
presidents = []
assert my_function(presidents) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 22: Rugby betting scandal

Exercise Description

In New Zealand, a rugby betting scandal has erupted. Some players have placed bets on certain matches despite the rules prohibiting it. The rules forbid betting from any **illegal** platform, and there is also a maximum amount of money that can be spent on legal platforms.

The federation has gathered a list `bets` of bets placed by players, where each element has the format `'name-bet-illegal'`:

- `name` is the name of the rugby player,
- `bet` is the amount wagered (a float),
- `illegal` is `0` if the site was legal, and `1` otherwise.

Write a function `my_function(bets, player)` that returns `True` if the player should be **disqualified**, and `False` otherwise.

A player is disqualified if:

- they placed **any** bet on an illegal platform, or
- the **total** amount they bet on legal platforms exceeds 500.

Your solution

```
def my_function(bets, player):
    total = 0
    for bet in bets:
        bet = bet.split('-')
        if bet[0] == player:
            if bet[2] == '1':
                return True
            else:
                total += float(bet[1])
    if total > 500:
        return True
    return False
```

Test your solution

```python
# Test case 1
bets = ['Beans-30.5-0', 'Beans-20.4-1', 'Tonal-50.5-1', 'Tonal-100.5-0', 'TheShara-21
player = 'Beans'
assert my_function(bets, player) is True

# Test case 2
player = 'Tonal'
assert my_function(bets, player) is True

# Test case 3
player = 'TheShara'
assert my_function(bets, player) is False

# Test case 4
player = 'Mikasa'
assert my_function(bets, player) is False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 23: Subtract digit sum

## Exercise Description

Write a function `subtract_digits(n)` that, given an integer number `n`, returns the number `d` obtained by **subtracting from `n` the sum of its digits**.

Hint: convert `n` into a string and work on each character, which corresponds to a digit.

Example: if `n = 12`, the function should return `9` because `12 - 1 - 2 = 9`.

## Your solution

```
def subtract_digits(n):
    n_s = str(n)
    for digit in n_s:
        n = n - int(digit)
    return n
```

## Test your solution

```
# Test case 1 (Example)
assert subtract_digits(12) == 9

# Test case 2
assert subtract_digits(1038) == 1026

# Test case 3
```

```python
assert subtract_digits(0) == 0

# Test case 4
assert subtract_digits(1199) == 1179
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 24: Sum larger or equal 3

Exercise Description

Write a function `my_function(L)` that, given a list `L` of strings, returns the **sum of the lengths** of the strings whose length is **greater than or equal to 3**.

If there are no items in the list or no strings of length ≥ 3, the function should return `0`.

Your solution

```python
def my_function(L):
    s = 0
    for string in L:
        if len(string) >= 3:
            s = s + len(string)
    return s
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(['luiss', 'row', 'a', 'bx']) == 8

# Test case 2
assert my_function(['a', 'a', 'a', 'a']) == 0

# Test case 3
assert my_function(['aa', 'aaaa', 'aaa', 'aaa']) == 10

# Test case 4
assert my_function([]) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

[🔗 Visualize YOUR solution in Python Tutor]

---

# Exercise 25: Sum of Even Squares

Exercise Description

Write a function `my_function(L)` that, given a list `L` of numbers, returns the **sum of the squares of the even numbers** contained in `L`.

If there are no items or no even numbers in the list, the function should return `0`.

Your solution

```
def my_function(L):
    # check if the list is empty
    if L == []:
        return 0
    # even numbers
    even_numbers = [x for x in L if x % 2 == 0]
    if even_numbers == []:
        return 0
    # square of even numbers
    return sum([x ** 2 for x in even_numbers])
```

Test your solution

```python
# Test case 1 (Example)
assert my_function([2, 13, 8, 5]) == 68

# Test case 2
assert my_function([1, 3, 5, 7]) == 0

# Test case 3
assert my_function([3, 101, 9, 7, 4]) == 16

# Test case 4
assert my_function([]) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 26: Sum of all the odd

Exercise Description

You have an integer number `n` as input. Define a function `all_odd(n)` that returns the **sum of all odd integer numbers between 1 and `n`**, `n` included.

Your solution

```python
def all_odd(n):
    c = 0
    for i in range(1, n + 1):
        if i % 2 == 0:
            pass
        else:
            c = c + i
    return c
```

Test your solution

```python
# Test case 1 (Example)
assert all_odd(7) == 16

# Test case 2
assert all_odd(6) == 9

# Test case 3
assert all_odd(100) == 2500

# Test case 4
assert all_odd(1000 * 1000) == 250000000000
```

### Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 27: Valid votes

### Exercise Description

You are given a list of votes `votes` and a candidate name `candidate`. Each vote is represented as a string in the format `'candidate-valid_vote'`, where `valid_vote` is either `0` or `1` (`1` means the vote is valid).

Write a function `my_function(votes, candidate)` that **counts how many valid votes** (with `valid_vote == '1'`) the given candidate received and returns that count. The candidate may have received 0 votes.

### Your solution

```
def my_function(votes, candidate):
    valid_votes = 0
```

```
    for vote in votes:
        if vote.split('-')[0] == candidate and vote.split('-')[1] == '1':
            valid_votes += 1
    return valid_votes
```

Test your solution

```
# Test case 1
votes = ['A-1', 'A-0', 'B-1', 'B-1', 'C-0', 'A-1', 'C-1', 'D-0']
assert my_function(votes, 'A') == 2

# Test case 2
votes = ['A-0', 'B-1', 'C-1', 'D-1', 'E-0', 'F-1', 'G-1', 'H-0']
assert my_function(votes, 'H') == 0

# Test case 3
votes = ['A-1', 'B-1', 'C-1', 'D-1', 'E-1', 'F-1', 'G-1', 'H-1']
assert my_function(votes, 'A') == 1

# Test case 4
votes = ['A-0', 'B-0', 'C-0', 'D-0', 'E-0', 'F-0', 'G-0', 'H-0']
assert my_function(votes, 'A') == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 28: Vowels

Exercise Description

You have a string `a` as input, along with two integer numbers `s` and `e`. Define a function `my_function(a, s, e)` that returns **how many vowels** are in the substring starting at position `s` and ending at position `e` (both included).

Consider the vowels `a`, `e`, `i`, `o`, `u` in both lowercase and uppercase.

Your solution

```
def my_function(a, s, e):
    a = a[s: e + 1]
    a = a.lower()
    c = 0
    for letter in a:
        if letter == 'a' or letter == 'e' or letter == 'i' or letter == 'o' or letter
            c = c + 1
    return c
```

Test your solution

```python
# Test case 1 (Example)
a = 'I study at Luiss'
assert my_function(a, 0, 15) == 5

# Test case 2
a = 'aaccccoa'
assert my_function(a, 0, 7) == 4

# Test case 3
a = 'AAAAmmm'
assert my_function(a, 4, 6) == 0

# Test case 4
a = 'xyz rtw lpq'
assert my_function(a, 1, 7) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 29: Aggregate Address

Exercise Description

Write a function `my_function(addresses)` that aggregates addresses by town.

`addresses` is a list of tuples, where each tuple has two string elements: `(town, street_name)`. For example, it could be `addresses = [('Roma', 'Via Panama'), ('Roma', 'Viale Romania'), ('Milano', 'Corso Sempione')]`.

The function shall return a dictionary that maps each town to the number of addresses in that town from the input list. If the input list is empty, the function should return an empty dictionary.

Your solution

```
def my_function(addresses):
    aggregator = {}
    # do not iterate if list is empty
    if len(addresses) == 0:
        return aggregator
    # otherwise, do
    for item in addresses:
        city = item[0]
        # address = item[1]
        if city in aggregator:
            aggregator[city] += 1
        else:
            aggregator[city] = 1
    return aggregator
```

Test your solution

```
# Test case 1 (Example)
a = my_function([('Roma', 'Via Panama'), ('Roma', 'Viale Romania'), ('Milano', 'Corso
assert a == {'Roma': 2, 'Milano': 1}

# Test case 2
a = my_function([('Londra', 'Via Panama'), ('Milano', 'Viale Romania'), ('Milano', 'C
assert a == {'Londra': 1, 'Milano': 2}

# Test case 3
a = my_function([])
assert a == {}

# Test case 4
a = my_function([('Firenze', 'Via Panama'), ('Como', 'Viale Romania'), ('Napoli', 'Co
assert a == {'Firenze': 1, 'Como': 1, 'Napoli': 1}
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 30: Dict Game 1

Exercise Description

Let `D` be a dictionary whose keys are words (strings) and whose corresponding values are integer numbers. Further, let `L` be a list of words.

Write a function `my_function(D, L)` that, given as input a dictionary `D` and a list `L`, returns the sum of the lengths of the words in the dictionary that satisfy the following requirements:

- The word must be present in the list
- The word must contain at least 1 uppercase character
- The value associated with the word must be an even number

If the input list is empty, the function must return `0`.

Your solution

```
def checkUppercase(word):
    uppercase = "ABCDEFGHILMNOPQRSTUVZ"
    for letter in uppercase:
      if letter in word:
        return True
    return False

def my_function(D,L):
    s = 0
    for el in L:
```

```python
        if el in D:
            if checkUppercase(el) and D[el] % 2 ==0:
                s += len(el)
    return s
```

Test your solution

```python
# Test case 1 (Example)
D = {"Sara" : 8, "Mara" : 3}
L = ["Alessio", "Sara", "Giorgio", "Mara"]
result = my_function(D, L)
assert result == 4

# Test case 2
D = {"Sara" : 8, "ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "ilaria"]
result = my_function(D, L)
assert result == 4

# Test case 3
D = {"Sara" : 8, "Ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "Ilaria"]
result = my_function(D, L)
assert result == 10

# Test case 4
D = {"Sara" : 8, "Ilaria" : 7}
L = []
result = my_function(D, L)
assert result == 0
```

### Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

[🔗 Visualize YOUR solution in Python Tutor]

---

# Exercise 31: Dict Game 2

### Exercise Description

Let `D` be a dictionary whose keys are words (strings) and whose corresponding values are integer numbers. Further, let `L` be a list of words.

Write a function `my_function(D, L)` that, given as input a dictionary `D` and a list `L`, returns the sum of the lengths of the words in the dictionary that satisfy the following requirements:

- The word must be present in the list
- The word must contain at least 1 uppercase character
- The value associated with the word must be an even number

If the input list is empty, the function must return 0.

Your solution

```python
def my_function(D, L):
    # if the input list is empty, return 0
    if L == []:
        return 0

    total_length = 0
    for word in L:
        # the word must be present in the dictionary
        if word in D:
            # the word must contain at least one uppercase character
            if any(ch.isupper() for ch in word):
                # the associated value must be even
                if D[word] % 2 == 0:
                    total_length += len(word)

    return total_length
```

Test your solution

```python
# Test case 1 (Example)
D = {"Sara" : 8, "Mara" : 3}
L = ["Alessio", "Sara", "Giorgio", "Mara"]
result = my_function(D, L)
assert result == 4

# Test case 2
D = {"Sara" : 8, "ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "ilaria"]
```

```python
result = my_function(D, L)
assert result == 4

# Test case 3
D = {"Sara" : 8, "Ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "Ilaria"]
result = my_function(D, L)
assert result == 10

# Test case 4
D = {"Sara" : 8, "Ilaria" : 7}
L = []
result = my_function(D, L)
assert result == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 32: Dict Game 3

Exercise Description

Let `D` be a dictionary whose keys are words (strings) and whose corresponding values are dates. Further, let `L` be a list of words and let `A` and `B` be two `datetime.date` objects where `A < B` (or `A` comes before `B`).

Write a function `my_function(D, L, A, B)` that, given as input a dictionary `D`, a list `L` and the two `date` objects `A` and `B`, returns the number of words (keys) in the dictionary that satisfy the following requirements:

- The word must be present in the list
- The word must contain at least one uppercase letter
- The date associated with the word must be between `A` and `B`

If the input list is empty, the function must return `0`.

**Hint**: Remember that it is possible to use the common operators for comparison (e.g., <, >, ==) on `date` objects.

**Recall**: To use the `datetime` module in your code, please make sure to import it before the `def` of your function.

Your solution

```
import datetime

def checkUppercase(word):
    uppercase = "ABCDEFGHILMNOPQRSTUVZ"
```

```
    for letter in uppercase:
      if letter in word:
        return True
    return False

def my_function(D, L, A, B):
    count = 0

    if L == []:     # check if the input list is empty: if so, return 0
        return 0

    for key in D.keys():
        if key in L:                    # if the word is in the list
            if checkUppercase(key):     # if the word has at least one uppercase lett
                if A <= D[key] <= B:    # if the date is between A and B
                    count += 1
    return count
```

Test your solution

```
# Test case 1 (Example)
import datetime
D = {
    "apple": datetime.date(2023, 3, 15),
    "Banana": datetime.date(2023, 4, 10),
    "Orange": datetime.date(2022, 12, 5),
    "Grapes": datetime.date(2023, 5, 20),
}
L = ["apple", "Banana", "Cherry", "Grapes"]
A = datetime.date(2023, 1, 1)
B = datetime.date(2023, 6, 30)
```

```python
result = my_function(D, L, A, B)
assert result == 2

# Test case 2
D = {
    "Hello": datetime.date(2023, 3, 15),
    "World": datetime.date(2023, 4, 10),
    "Python": datetime.date(2022, 12, 5),
    "Programming": datetime.date(2023, 5, 20),
}
L = ["Hello", "Programming", "AI", "MachineLearning"]
A = datetime.date(2023, 1, 1)
B = datetime.date(2023, 6, 30)
result = my_function(D, L, A, B)
assert result == 2

# Test case 3
D = {
    "Car": datetime.date(2023, 3, 15),
    "Bus": datetime.date(2023, 4, 10),
    "Bicycle": datetime.date(2022, 12, 5),
    "Train": datetime.date(2023, 5, 20),
}
L = ["Car", "Bus", "Bicycle", "Train"]
A = datetime.date(2023, 3, 1)
B = datetime.date(2023, 4, 30)
result = my_function(D, L, A, B)
assert result == 2

# Test case 4
D = {
```

```python
    "Sun": datetime.date(2023, 3, 15),
    "Moon": datetime.date(2023, 4, 10),
    "Stars": datetime.date(2022, 12, 5),
    "Planets": datetime.date(2023, 5, 20),
}
L = ["Sun", "Moon", "Stars", "Planets"]
A = datetime.date(2024, 1, 1)
B = datetime.date(2024, 6, 30)
result = my_function(D, L, A, B)
assert result == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 33: Dict Game 4

Exercise Description

Let `D` be a dictionary whose keys are words (strings) and whose corresponding values are integer numbers. Further, let `L` be a list of words.

Write a function `my_function(D, L)` that, given as input a dictionary `D` and a list `L`, returns the sum of the lengths of the words in the dictionary that satisfy the following requirements:

- The word must be present in the list
- The word must contain at least 1 uppercase character
- The value associated with the word must be an even number

If the input list is empty, the function must return 0.

Your solution

```
def my_function(D, L):
    s = 0
    # Iterate over the dictionary entries
    for word, value in D.items():
        # Check all required conditions
        if (
            word in L and
            any(ch.isupper() for ch in word) and
            value % 2 == 0
        ):
            s += len(word)
    return s
```

Test your solution

```
# Test case 1 (Example)
D = {"Sara" : 8, "Mara" : 3}
```

```
L = ["Alessio", "Sara", "Giorgio", "Mara"]
result = my_function(D, L)
assert result == 4

# Test case 2
D = {"Sara" : 8, "ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "ilaria"]
result = my_function(D, L)
assert result == 4

# Test case 3
D = {"Sara" : 8, "Ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "Ilaria"]
result = my_function(D, L)
assert result == 10

# Test case 4
D = {"Sara" : 8, "Ilaria" : 7}
L = []
result = my_function(D, L)
assert result == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 34: Dict Game 5

Exercise Description

Let D be a dictionary whose keys are words (strings) and whose corresponding values are integer numbers. Further, let L be a list of words.

Write a function my_function(D, L) that, given as input a dictionary D and a list L, returns the sum of the lengths of the words in the dictionary that satisfy the following requirements:

- The word must be present in the list
- The word must contain at least 1 uppercase character
- The value associated with the word must be an even number

If the input list is empty, the function must return 0.

Your solution

```
def my_function(D, L):
    s = 0
    # Iterate over the dictionary entries
    for word, value in D.items():
        # Check all required conditions
        if (
```

```python
            word in L and
            any(ch.isupper() for ch in word) and
            value % 2 == 0
        ):
            s += len(word)
    return s
```

Test your solution

```python
# Test case 1 (Example)
D = {"Sara" : 8, "Mara" : 3}
L = ["Alessio", "Sara", "Giorgio", "Mara"]
result = my_function(D, L)
assert result == 4

# Test case 2
D = {"Sara" : 8, "ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "ilaria"]
result = my_function(D, L)
assert result == 4

# Test case 3
D = {"Sara" : 8, "Ilaria" : 4}
L = ["Alessio", "Sara", "Giorgio", "Mara", "Ilaria"]
result = my_function(D, L)
assert result == 10

# Test case 4
D = {"Sara" : 8, "Ilaria" : 7}
L = []
```

```
result = my_function(D, L)
assert result == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 35: Encrypt

Exercise Description

Define a function `my_function(S, D)` that decrypts a text `S`, previously encrypted using a simple cipher called "letter substitution cipher". `D` is a dictionary that maps encrypted characters to decrypted characters. If a character of string `S` is a key of the dictionary, it shall be substituted by its corresponding dictionary value, otherwise it should not change. The function shall return the decrypted text.

Your solution

```python
def my_function(S, D):
    result = ""
    for el in S:
        if el in D:
            result += D[el]
        else:
            result += el
    return result
```

Test your solution

```python
# Test case 1 (Example)
a = my_function("hvssm", {"v": "e", "t": "w", "s": "l", "m": "o"})
assert a == "hello"

# Test case 2
a = my_function("vtmx", {"v": "e", "t": "w", "s": "l", "m": "o"})
assert a == "ewox"

# Test case 3
a = my_function("xxxxqqqq", {"v": "e", "t": "w", "s": "l", "m": "o"})
assert a == "xxxxqqqq"

# Test case 4
a = my_function("", {"v": "e", "t": "w", "s": "l", "m": "o"})
assert a == ""
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 36: Football League

Exercise Description

Write a function `my_function(matches)` that computes the points of the teams taking part in a football league.

Parameter `matches` is a list of tuples representing matches, such as: `('Roma', 'Juventus', 3, 2)`.

Each tuple has the following four elements:

1. the name of the home team
2. the name of the away team
3. the number of goals of the home team
4. the number of goals of the away team

The function must compute a dictionary that maps each team to its total number of points in the league, where teams are awarded 3 points for a win, 1 point for a draw and no points for a loss. The function shall then return the difference between the maximum number of points and the minimum number of points achieved by teams that took part in the league.

**Back-of-the-envelope example**: if `matches` is

```
[('Roma', 'Juventus', 3, 2), ('Milan', 'Juventus', 1, 1), ('Roma', 'Milan', 5,
1), ('Juventus', 'Milan', 2, 1)]
```

- Roma gets in total 6 points, i.e., 3 points from each of its two matches.
- Juventus gets in total 4 points: 1 point from the draw with Milan, and 3 points from the winning match with Milan.
- Milan gets only 1 point from the draw with Juventus.

The function shall thus return 5 = 6 - 1.

Your solution

```python
def my_function(matches):
    teams = []
    for match in matches:
        teams.append(match[0])
        teams.append(match[1])

    D = {team: 0 for team in teams}

    for match in matches:
        team1 = match[0]
```

```python
        team2 = match[1]
        score1 = match[2]
        score2 = match[3]
        if score1==score2:
            D[team1] += 1
            D[team2] += 1
        elif score1 > score2:
            D[team1] += 3
            D[team2] += 0
        elif score1 < score2:
            D[team1] += 0
            D[team2] += 3

    return max(D.values()) - min(D.values())
```

Test your solution

```python
# Test case 1 (Example)
matches = [('Roma', 'Juventus', 3, 2), ('Milan', 'Juventus', 1, 1), ('Roma', 'Milan',
assert my_function(matches) == 5

# Test case 2
matches = [('Roma', 'Juventus', 3, 3)]
assert my_function(matches) == 0

# Test case 3
matches = [('Roma', 'Juventus', 3, 3), ('Milan', 'Juventus', 2, 1)]
assert my_function(matches) == 2

# Test case 4
matches = [
```

```python
    ('Barcelona', 'Real Madrid', 2, 2),
    ('Manchester United', 'Manchester City', 1, 3),
    ('Liverpool', 'Chelsea', 2, 1),
    ('Bayern Munich', 'Borussia Dortmund', 3, 2),
    ('Paris Saint-Germain', 'AS Monaco', 1, 1),
    ('Atletico Madrid', 'Sevilla', 0, 2),
    ('AC Milan', 'Inter Milan', 2, 2),
    ('Juventus', 'Napoli', 1, 0),
    ('Ajax', 'PSV Eindhoven', 3, 1),
    ('Porto', 'Benfica', 2, 2)
]
assert my_function(matches) == 3
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 37: Morse Code

Exercise Description

Define a function `my_function(s, morse_dict)` that decodes a string `s`, representing a word encoded in Morse Code.

In Morse Code:

- each letter is encoded as a sequence of dashes and dots. For example, letter `C` is encoded as `-.-.` (dash dot dash dot)
- encoded letter are separated by a `/` symbol (slash)

The function shall return the decoded string or an empty string in case `s` is empty.

Parameter `morse_dict` is a dictionary whose keys are morse-encoded letters: keys are mapped to plaintext (decoded) letters.

Your solution

```
def my_function(s, morse_dict):
    """
    Define a function my_function(s, morse_dict) that decodes a string s, representin

    In Morse Code:
    * each letter is encoded as a sequence of dashes and dots. For example, letter C
    * encoded letter are separated by a / symbol (slash)
    Parameter morse_dict is a dictionary whose keys are morse-encoded letters: keys a
    The function shall return the decoded string o an empty string in case s is empty
    """
    if s == '':
        return s
```

```
    splitted = s.split('/')
    decoded = ''
    for item in splitted:
        decoded = decoded + morse_dict[item]
    return decoded
```

Test your solution

```
# Test case 1 (Example)
morse_code = {
 '.-': 'A',
 '-...': 'B',
 '-.-.': 'C',
 '-..': 'D',
 '.': 'E',
 '..-.': 'F',
 '--.': 'G',
}
word_in_morse = '..-./.-/-.-./.'
word = my_function(word_in_morse, morse_code)
assert word == 'FACE'

# Test case 2
morse_code = {
 '.-': 'A',
 '-...': 'B',
 '-.-.': 'C',
 '-..': 'D',
 '.': 'E',
 '..-.': 'F',
 '--.': 'G',
```

```python
}
word_in_morse = '--././--./..-.'
word = my_function(word_in_morse, morse_code)
assert word == 'GEGF'

# Test case 3
morse_code = {
 '.-': 'A',
 '-...': 'B',
 '-.-.': 'C',
 '-..': 'D',
 '.': 'E',
 '..-.': 'F',
 '--.': 'G',
}
word_in_morse = ''
word = my_function(word_in_morse, morse_code)
assert word == ''

# Test case 4
morse_code = {
 '.-': 'A',
 '-...': 'B',
 '-.-.': 'C',
 '-..': 'D',
 '.': 'E',
 '..-.': 'F',
 '--.': 'G',
}
word_in_morse = '-.-./-.-./-.-./-.-./-.-./-.-.'
```

```
word = my_function(word_in_morse, morse_code)
assert word == 'CCCCCC'
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 38: Musical Albums

Exercise Description

Write a Python function called `my_function(L, S)` that takes as input a list `L` and a string `S`.

List `L` is a list of musical albums, where each album is described as a dictionary with 5 fields:

- `"title"` (string): the title of the album
- `"artist"` (string): the artistIf there are no albums with such genre in `L`, the function must return `-1`. If `L` is empty, the function must return `0`.
- `"genre"` (string): its genre
- `"rating"` (float): the rating for the album (scale from 1 to 5)The function must return the average rating calculated amongst all albums in `L` whose genre is `S`.
- `"year"` (integer): the release year

whereas `S` is a string which corresponds to a genre.

Your solution

```python
def my_function(L, S):
    # check if L is empty, if so return 0
    if L == []:
        return 0
    # take all albums from L that have genre S
    subset = [album for album in L if album["genre"] == S]
    # check if subset is empty, if so return -1
    if subset == []:
        return -1
    # take the average rating of all albums in subset
    avg_rating = sum([album["rating"] for album in subset]) / len(subset)
```

```
    # return the average rating
    return avg_rating
```

Test your solution

```
# Test case 1 (Example)
S = 'Grunge'
L = [
    {
        "title": "Nevermind",
        "artist": "Nirvana",
        "genre": "Grunge",
        "year": 1991,
        "rating": 4.5
    },
    {
        "title": "London Calling",
        "artist": "The Clash",
        "genre": "Punk",
        "year": 1979,
        "rating": 4.7
    },
    {
        "title": "Unknown Pleasures",
        "artist": "Joy Division",
        "genre": "Dark",
        "year": 1979,
        "rating": 4.6
    },
    {
        "title": "Siamese Dream",
```

```python
        "artist": "The Smashing Pumpkins",
        "genre": "Grunge",
        "year": 1993,
        "rating": 4.4
    }
]
assert my_function(L, S) == 4.45

# Test case 2
S = "Parento"
L = [
    {
        "title": "Nevermind",
        "artist": "Nirvana",
        "genre": "Grunge",
        "year": 1991,
        "rating": 4.5
    },
    {
        "title": "London Calling",
        "artist": "The Clash",
        "genre": "Punk",
        "year": 1979,
        "rating": 4.7
    },
    {
        "title": "Unknown Pleasures",
        "artist": "Joy Division",
        "genre": "Dark",
        "year": 1979,
        "rating": 4.6
```

```
    },
    {
        "title": "Siamese Dream",
        "artist": "The Smashing Pumpkins",
        "genre": "Grunge",
        "year": 1993,
        "rating": 4.4
    }
]
assert my_function(L, S) == -1

# Test case 3
S = "Dark"
L = [
    {
        "title": "Nevermind",
        "artist": "Nirvana",
        "genre": "Grunge",
        "year": 1991,
        "rating": 4.5
    },
    {
        "title": "London Calling",
        "artist": "The Clash",
        "genre": "Punk",
        "year": 1979,
        "rating": 4.7
    },
    {
        "title": "Unknown Pleasures",
        "artist": "Joy Division",
```

```
        "genre": "Dark",
        "year": 1979,
        "rating": 4.6
    },
    {
        "title": "Siamese Dream",
        "artist": "The Smashing Pumpkins",
        "genre": "Grunge",
        "year": 1993,
        "rating": 4.4
    }
]
assert my_function(L, S) == 4.6

# Test case 4
S = "Rock"
L = []
assert my_function(L, S) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 39: Planet 4z Mission Cost

Exercise Description

4z is a pacific planet where all of its inhabitants live peacefully. The only rule they must obey is that their names must contain the letter `'z'` or they have to be at most 4 letters long. It's 11-11-2222 when planet Earth, close to extinction, sends an SOS to planet 4z. Attached to the SOS, there's a dictionary of earthlings whose structure is:

```
{
"name of earthling 1" : (its age, its weight),
"name of earthling 2" : (its age, its weight),
...
}
```

Help the aliens from planet 4z to calculate the overall cost of the rescue by writing a function `my_function(humans)` that, given the dictionary of earthlings, returns the mission cost.

The cost of a space shuttle for a human whose weight is more than 80kg is 25eth, otherwise it is 15eth. Furthermore, for humans older than 80 years old, there is an additional 5eth charge for carrying medical equipment.

Your solution

```
def my_function(humans):
    cost = 0
    for key in humans:
        if len(key) <= 4 or "z" in key:
            age, weight = humans[key]
```

```
            cost += 15
            if weight > 80:
                cost += 10
            if age > 80:
                cost += 5
    return cost
```

Test your solution

```
# Test case 1 (Example)
a = {'giorgio' : (35, 88), 'ind' : (20, 15), 'alessioz' : (90, 67) }
assert my_function(a) == 35

# Test case 2
a = {'giorgioz' : (35, 88), 'ind' : (20, 15), 'alessioz' : (90, 67) }
assert my_function(a) == 60

# Test case 3
a = {'giorgio' : (35, 88), 'indro' : (20, 15), 'alessio' : (90, 67) }
assert my_function(a) == 0

# Test case 4
a = { }
assert my_function(a) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 40: Shopping By Quantity

Exercise Description

# Shopping by Quantity

Write a function `my_function(prices, cart)` that computes the total price to be paid for items in a shopping cart.

Parameter `prices` is a list of tuples `(item, price)`, where `item` is a string and `price` is a number: for example, `prices` could be `[('cookies', 4.2), ('pasta', 3.0), ('eggs', 3.2), ('milk', 2.3)]`.

Parameter `cart` is a dictionary that maps items to the number of sold units, for example `{'pasta': 4, 'milk': 2,  'cookies': 3}`.

You can assume that each key in the cart dictionary appears in the prices list.

**Back-of-the-envelope example**: `my_function([('cookies', 4.2), ('pasta', 3.0), ('eggs', 3.2), ('milk', 2.3)], {'pasta': 4, 'milk': 2,  'cookies': 3})` should return 29.2 because 29.2 = 4.2*3 + 3.0*4 + 2.3*2.

The function shall return the total cost of items in the cart dictionary. If there are no items in the cart, the function should return 0.

Your solution

```
def my_function(prices, cart):
    if len(cart) == 0:
        return 0

    items = [prices[i][0] for i in range(len(prices))]

    s = 0
    for k, v in cart.items():
        i = items.index(k)
        s = s + v * prices[i][1]
    return s
```

Test your solution

```
# Test case 1 (Example)
a = my_function([('cookies', 4.2), ('pasta', 3.0), ('eggs', 3.2), ('milk', 2.3)], {'p
assert abs(a - 29.2) < 0.0001
```

```
# Test case 2
a = my_function([('cookies', 4.2), ('orange', 1.0), ('eggs', 3.2), ('milk', 2.3)], {'
assert abs(a - 8.6) < 0.0001

# Test case 3
a = my_function([('cookies', 4.2), ('orange', 1.0), ('bacon', 3.2), ('milk', 2.3)], {
assert abs(a - 35.2) < 0.0001

# Test case 4
a = my_function([('cookies', 4.2), ('orange', 1.0), ('bacon', 3.2), ('milk', 2.3)], {
assert abs(a - 0.0) < 0.0001
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 41: Shopping By Prices

Exercise Description

# Shopping by Prices

Write a function `my_function(shopping_cart, prices)` that computes the total price to be paid for items in a shopping cart.

Parameter `shopping_cart` is a list of tuples `(item, quantity)`, where `item` is a string and `quantity` is a number: for example, `shopping_cart` could be `[('pasta', 3), ('milk', 2), ('cookies', 4)]`.

Parameter `prices` is a dictionary that maps items to their unit price, for example `{'pasta': 0.69, 'milk': 1.19, 'eggs': 0.79, 'bread': 1.99, 'cookies': 2.49}`.

You can assume that each item in the shopping list is a key in the `prices` dictionary.

**Back-of-the-envelope example**: `my_function([('pasta', 3), ('milk', 2), ('cookies', 4)], {'pasta': 0.69, 'milk': 1.19, 'eggs': 0.79, 'bread': 1.99, 'cookies': 2.49})` should return 14.41 because 14.41 = 0.69*3 + 1.19*2 + 2.49*4.

The function shall return the total cost of items in the shopping cart. If there are no items in the cart, the function should return `0`.

Your solution

```
def my_function(shopping_cart, prices):
    total = 0
    if len(shopping_cart) == 0:
        return total
```

```
    for object, quantity in shopping_cart:
        total = total + quantity * prices[object]
    return total
```

Test your solution

```
# Test case 1 (Example)
a = my_function([('pasta', 3), ('milk', 2), ('cookies', 4)], {'pasta': 0.69, 'milk':
assert a == 14.41

# Test case 2
a = my_function([('orange', 1), ('cookies', 10)], {'cookies': 0.69, 'milk': 1.19, 'eg
assert a == 9.39

# Test case 3
a = my_function([('orange', 1)], {'cookies': 0.69, 'milk': 1.19, 'eggs': 0.79, 'bread
assert a == 2.49

# Test case 4
a = my_function([], {'cookies': 0.69, 'milk': 1.19, 'eggs': 0.79, 'bread': 1.99, 'ora
assert float(a) == 0.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 42: Wallet

Exercise Description

Write a function `my_function (wallet, price)`that computes the total value inside the wallet in Euro.

Parameter `wallet` is a list of tuples `(currency, quantity)`, where `currency` is a string and `quantity` is a number: for example, `wallet` could be `[("gbp", 3), ("dollar", 2), ("gbp", 4)]`.

Parameter `price` is a dictionary that maps the conversion of different currencies in euro (1 euro) , for example `{"krona": 7.44, "dollar": 1.22, "gbp": 0.89, "real": 6.65}`.

You can assume that the currency in each item in the wallet appears as a key in the price dictionary.

The function shall return the total value in euro inside the wallet. If there are no items in the wallet, the function should return 0.

Back-of-the-envelope example: `my_function([("gbp", 2), ("dollar", 2)], {"krona": 7.44 , "dollar": 1.22, "gbp": 0.89, "real": 6.65})` should return 4.22 = 2*0.89 + 2*1.22.

Your solution

```
def my_function(wallet, price):
    if wallet == []:
        return 0
    else:
        s = 0
        for currency, quantity in wallet:
            s += quantity * price[currency]
        return s
```

Test your solution

```
# Test case 1 (Example)
a = my_function([("gbp", 2), ("dollar", 2)], {"krona": 7.44, "dollar": 1.22, "gbp": 0
assert a == 4.22

# Test case 2
a = my_function([("dollar", 2)], {"krona": 7.44, "dollar": 1.22, "gbp": 0.89, "real":
assert a == 2.44

# Test case 3
a = my_function([("krona", 1), ("dollar", 2)], {"krona": 7.44, "dollar": 1.22, "gbp":
assert a == 9.88

# Test case 4
a = my_function([], {"krona": 7.44, "dollar": 1.22, "gbp": 0.89, "real": 6.65})
assert a == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 43: Automatic writing

Exercise Description

Write a function `my_function(a, b)` that takes two lists:

- `a` is a list of characters, for example `['b', 'e', 'l', 'a']`
- `b` is a list of integers, for example `[1, 1, 2, 1]`

The function should behave as follows:

- If the two lists `a` and `b` have different lengths, the function returns `-1`.
- If there is at least one negative value in `b`, the function returns `-1`.
- Otherwise, the function returns a string containing the characters in `a` repeated as many times as the number specified in the corresponding position in `b`.

For example, `my_function(['b','e','l','a'], [1,1,2,1])` should return `'bella'`.

Your solution

```
def my_function(a, b):
    # check whether the two lists have different length: if so, return -1
    if len(a) != len(b):
        return -1
    # check whether there are some negative elements in b: if so, return -1
    for n in b:
        if n < 0:
            return -1
    # if no returns have been triggered so far, then create the proper string
    s = ''
    for i in range(len(a)):
        s += a[i] * b[i]
    return s
```

Test your solution

```
# Test case 1 (Example)
assert my_function(['b','e','l','a'], [1,1,2,1]) == 'bella'

# Test case 2
```

```python
assert my_function(['a','b','c'], [3,2,0]) == 'aaabb'

# Test case 3
assert my_function(['a','b','c'], [3,2,-1]) == -1

# Test case 4
assert my_function(['a','b','c'], [3,2,2,2]) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 44: List concatenations even n

Exercise Description

Write a function `my_function(lst, n)` which, given as input a list of strings `lst` and a number `n`, returns a string that concatenates all the strings in the list that:

- have length strictly greater than n, and
- have an even length.

If none of the strings satisfy the requirements, the function must return the string 'no'.

If the input list is empty, the function must return -1.

Your solution

```
def my_function(lst, n):
    if len(lst) == 0:
        return -1
    result = ""
    for el in lst:
        if len(el) > n and len(el) % 2 == 0:
            result += el
    if len(result) == 0:
        return "no"
    else:
        return result
```

Test your solution

```
# Test case 1 (Example)
assert my_function(["chirus", "sam", "z", "alessio martino"], 4) == "chirus"

# Test case 2
assert my_function(["chirus", "samo", "zo", "alessio martino", "time"], 2) == "chirus

# Test case 3
```

```python
assert my_function(["chirus", "samo", "zo", "alessio martino", "time"], 22) == "no"

# Test case 4
assert my_function([], 4) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 45: Loop Game 1: prime-weighted sum

Exercise Description

Write a function `my_function(lst)` which takes as input a list of numbers `lst` and returns the sum of the numbers in the list, with the following rules:

- Prime numbers are worth double (for example, 7 is worth 14).
- If a `0` is encountered, the current sum is reset to `0`.
- If the number `-1` is encountered, the loop is interrupted and the function returns the sum obtained up to that point.

If the list is empty, the function should return `0`.

Your solution

```
def is_prime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True

def my_function(lst):
    result = 0
    for el in lst:
        if el == -1:
            break
        elif el == 0:
            result = 0
        elif is_prime(el):
            result += el * 2
        else:
            result += el
    return result
```

Test your solution

```python
# Test case 1 (Example)
assert my_function([4, 6, 8]) == 18

# Test case 2
assert my_function([99, 45, 18, 0, 7]) == 14

# Test case 3
assert my_function([40, 40, -1, 100, 7, 0, 12]) == 80

# Test case 4
assert my_function([]) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 46: Loop Game 2: character counter

Exercise Description

Write a function `my_function(lst)` which takes as input a list of strings `lst` and returns the total "value" of all the characters in the list. The rules are:

- Lowercase letters are worth 1 point each.
- Uppercase letters are worth 2 points each.
- If the character `'0'` is encountered, the current total is doubled (the character `'0'` itself does not add points).
- If the lowercase character `'z'` is encountered, the loop stops immediately and the function returns `-1`.

The function returns the final value according to these rules.

Your solution

```
def my_function(lst):
    capital = "ABCDEFGHILMNOPQRSTUVZ"
    result = 0
    for el in lst:
        for c in el:
            if c in capital:
                result += 2
            elif c == "0":
                result += result
            elif c == "z":
                return -1
            else:
```

```python
                    result += 1
    return result
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(["hello", "test", "hi"]) == 11

# Test case 2
assert my_function(["hEllo", "teSt", "hi"]) == 13

# Test case 3
assert my_function(["hEllo", "teSt", "0i"]) == 23

# Test case 4
assert my_function(["hEllz", "teSt", "0i"]) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 47: Loop Game 3: even numbers and powers

Exercise Description

Write a function `my_function(lst)` which takes as input a list of numbers `lst` and returns the sum of the numbers in the list, with the following rules:

- Even numbers less than 100 are worth double (for example, 50 is worth 100).
- If a `0` is encountered, the current sum is doubled.
- If the number `-1` is encountered, the loop is interrupted and the function returns the square of the current sum (`sum ** 2`).

If the list is empty, the function should return `0`.

Your solution

```
import math

def my_function(lst):
    result = 0
    for el in lst:
        if el % 2 == 0 and el < 100:
            result += el * 2
        elif el == 0:
            result += result
        elif el == -1:
            return math.pow(result, 2)
```

```
        else:
            result += el
    return result
```

Test your solution

```
# Test case 1 (Example)
assert int(my_function([1, 1, 1, 102])) == 105

# Test case 2
assert int(my_function([2, 2, 2, 102])) == 114

# Test case 3
assert int(my_function([])) == 0

# Test case 4
assert int(my_function([2, 2, -1, 102])) == 64
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 48: Loop Game 4: filtered strings count

Exercise Description

Write a function `my_function(lst, n)` which takes as inputs a list of strings `lst` and an integer `n` and returns the number of elements in the list that satisfy all the following conditions:

- the string length is **less than or equal to** `n`, and
- the string does **not** contain any uppercase characters.

If an empty string `""` is encountered in the list, the function must stop and return the count obtained up to that point.

Your solution

```
def checkUpper(w):
    capital = "ABCDEFGHILMNOPQRSTUVZ"
    for el in w:
        if el in capital:
            return True
    return False

def my_function(lst, n):
    count = 0
    for el in lst:
        if el == "":
            return count
```

```python
        if (not checkUpper(el)) and len(el) <= n:
            count += 1
    return count
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(["a", "ab", "abc"], 3) == 3

# Test case 2
assert my_function(["a", "ab", "abc"], 2) == 2

# Test case 3
assert my_function(["A", "aB", "abc"], 3) == 1

# Test case 4
assert my_function(["abcd", "ab", "A", "", "abc"], 3) == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 49: Merge words

Exercise Description

Write a function `my_function(a, b)` that adds two lists index-wise.

- `a` and `b` are lists whose elements are strings (or characters).
- The function returns a new list where each element is the concatenation of the elements from `a` and `b` in the same position.

If the lists do not have the same length, the function should return `-1`.

Your solution

```
def my_function(a, b):
    if len(a) != len(b):
        return -1
    res = [i + j for i, j in zip(a, b)]
    return res
```

Test your solution

```
# Test case 1 (Example)
assert my_function(['Pro','Ale','Mar'], ['f','ssio','tino']) == ['Prof', 'Alessio', '

# Test case 2
assert my_function(['Pro','Ale','Mar'], ['f','ssio']) == -1
```

```
# Test case 3
assert my_function(['Ta','va','gat'], ['nto',' la','ta']) == ['Tanto', 'va la', 'gatt

# Test case 4
assert my_function(['Pro','Ale'], ['nto','ssandro']) == ['Pronto', 'Alessandro']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 50: Password Encoder

Exercise Description

Write a function `my_function(s, letter)` that helps to build a password.

- `s` is a string.
- `letter` is a character and must be one of `['i', 'a', 's']`.

The function converts the string into a password with the following rules:

- If there are **no capital letters** in `s`, the function returns `-1`.
- If `letter` is **not** one of `'i'`, `'a'`, `'s'`, the function returns `-1`.
- If `letter` does not appear in `s`, the function returns `-1`.
- Otherwise, it returns a new string where every occurrence of `letter` is replaced by a special character according to this mapping:
    - `'i'` becomes `'!'`
    - `'a'` becomes `'@'`
    - `'s'` becomes `'$'`

All other characters remain unchanged.

Your solution

```
def my_function(s, letter):
    capital = "ABCDEFGHILMNOPQRSTUVZ"
    c = False
    for x in s:
        if x in capital:
            c = True
    if c is False:
        return -1
    if letter not in 'ais':
        return -1
    if letter not in s:
        return -1
    res = ""
```

```python
    for x in s:
        if x == letter:
            if letter == 'i':
                res += '!'
            elif letter == 'a':
                res += '@'
            elif letter == 's':
                res += '$'
        else:
            res += x
    return res
```

Test your solution

```python
# Test case 1 (Example)
assert my_function("Mammabuttalapasta", "a") == "M@mm@butt@l@p@st@"

# Test case 2
assert my_function("SerpentiSuonatoridiSonagli", "s") == -1

# Test case 3
assert my_function("CaniCattivi", "i") == "Can!Catt!v!"

# Test case 4
assert my_function("CaniCattivi", "q") == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code**

**before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 51: Planet 4z

Exercise Description

It is the year 2222 and the Earth is nearing its end. The inhabitants of planet 4z have decided to help the earthlings by sending escape ships. Each spacecraft can hold only one person.

On planet 4z there is an inviolable rule: the name of all its inhabitants must **contain the letter `'z'`** or be **at most 4 characters long**. Any humans who fail to meet these requirements will not be saved.

Write a function `my_function(inhabitants)` that, given as input a list of names of earthlings, returns the number of ships to be sent to planet Earth (i.e., the number of humans that satisfy the rule).

If none of the names meet the requirements, the function must return `-1`.

Your solution

```python
def my_function(inhabitants):
    ships = 0
    for el in inhabitants:
        if len(el) <= 4 or 'z' in el:
            ships += 1
    if ships == 0:
        return -1
    else:
        return ships
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(["chirus", "zzzzzzz"]) == 1

# Test case 2
assert my_function(["chirus"]) == -1

# Test case 3
assert my_function(["chirus", "sam", "z", "alessio martino"]) == 2

# Test case 4
assert my_function([]) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 52: Repeat

Exercise Description

Write a function `my_function(lst)` that takes a list of integers as input and returns a new list of integers.

The new list is built as follows:

- the element in position 0 is repeated 1 time,
- the element in position 1 is repeated 2 times,
- the element in position 2 is repeated 3 times, and so on.

However, you do **not** like the number `4`, therefore:

- you must **ignore** any element whose value is `4` in the output list, and
- you must **never** repeat an element **4 times** (i.e., when the position index + 1 is 4, skip that element).

Your solution

```python
def my_function(lst):
    res = []
    for i, val in enumerate(lst):
        i += 1
        if i == 4 or val == 4:
            continue
        for _ in range(i):
            res.append(val)
    return res
```

Test your solution

```python
# Test case 1 (Example)
assert my_function([1, 2, 3, 4]) == [1, 2, 2, 3, 3, 3]

# Test case 2
assert my_function([9, 8, 7, 6, 5]) == [9, 8, 8, 7, 7, 7, 5, 5, 5, 5, 5]

# Test case 3
assert my_function([4, 4, 4, 2, 1]) == [1, 1, 1, 1, 1]

# Test case 4
assert my_function([4, 4, 4, 4, 4, 4, 4]) == []
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 53: Reverse (almost) everything

Exercise Description

Write a function `my_function(lst, key)` that takes as input a list of words `lst` and a word `key`.

- If `key` is **not** in the list, the function should return `-1`.
- Otherwise, it should return a new list where **all words are reversed**, except for `key`, which remains unchanged (but stays in its original position).

Your solution

```
def my_function(lst, key):
    if key not in lst:
        return -1
    res = []
    for x in lst:
        if x != key:
            res.append(x[::-1])
        else:
```

```
            res.append(x)
    return res
```

Test your solution

```
# Test case 1 (Example)
assert my_function(['hello', 'world', 'python'], 'world') == ['olleh', 'world', 'noht

# Test case 2: key not present
assert my_function(['a', 'b', 'c'], 'x') == -1

# Test case 3: key at the beginning
assert my_function(['key', 'abc', 'def'], 'key') == ['key', 'cba', 'fed']

# Test case 4: key at the end
assert my_function(['abc', 'def', 'key'], 'key') == ['cba', 'fed', 'key']
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pytl
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 54: Sandwich

Exercise Description

Write a function `my_function(lst, k)` that returns a list of all the elements in `lst` that are **between two values** `k`.

That is, for each element in the list, you check whether the element before it and the element after it are both equal to `k`. If so, this element is included in the output.

Additional rules:

- If there are multiple occurrences of the same value that satisfy the condition, the output list should contain each such value **only once** (no duplicates).
- If `k` is not in `lst`, the function should return `-1`.

Your solution

```
def my_function(lst, k):
    if k not in lst:
        return -1
    res = []
    for i, v in enumerate(lst[:-1]):
        if v in res or i == 0:
            continue
        if (lst[i - 1] == k) and (lst[i + 1] == k):
```

```
            res.append(v)
    return res
```

Test your solution

```
# Test case 1 (Example)
assert my_function([2, 1, 2, 3, 4, 5, 2, 4, 2, 4, 2], 2) == [1, 4]

# Test case 2: k not present
assert my_function([2, 1, 2, 3, 4, 5, 2, 4, 2, 4, 2], 7) == -1

# Test case 3: repeated values
assert my_function([2, 2, 2, 3, 4, 5, 2, 4, 2, 4, 2], 2) == [2, 4]

# Test case 4
assert my_function([2, 2, 2, 2, 2, 2, 2, 3, 4, 3, 4, 3, 2, 3, 2], 2) == [2, 3]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 [Visualize YOUR solution in Python Tutor]()

---

# Exercise 55: Converting Time 1

Exercise Description

Write a function `my_function(K, L)` that is given an integer number `K` and a list of tuples `L`. Each tuple is a pair consisting of an integer number and a character `'m'` (minutes), `'h'` (hours), or `'s'` (seconds).

After converting hours and minutes to seconds, the function should return the position `i` of the tuple in list `L` such that the sum of seconds up to position `i` is **smaller than or equal to** `K`. Positions start from 0.

If the input list is empty or the number of seconds specified by the tuple in position 0 is larger than `K`, the function should return `-1`.

Example: `my_function(1000, [(2, 'm'), (25, 's'), (1, 'h'), (1, 's')])` should return 1, because 2 minutes = 120 seconds, 1 hour = 3600 seconds, and 120 + 25 = 145 < 1000, while 120 + 25 + 3600 = 3745 > 1000.

Your solution

```
def my_function(K, L):
    L_sec = []
    for i, unit in L:
        if unit == 's':
            L_sec.append(i)
        elif unit == 'm':
            L_sec.append(i * 60)
```

```
        elif unit == 'h':
            L_sec.append(i * 3600)

    if len(L) == 0 or L_sec[0] > K:
        return -1

    cumsum = [0 for _ in range(len(L))]
    for i in range(len(L)):
        cumsum[i] = sum(L_sec[: i + 1])

    TF = [item <= K for item in cumsum]

    if False in TF:
        return TF.index(False) - 1
    else:
        return len(L) - 1
```

Test your solution

```
# Test case 1 (Example)
assert my_function(1000, [(2, 'm'), (25, 's'), (1, 'h'), (1, 's')]) == 1

# Test case 2
assert my_function(121, [(2, 'm'), (25, 's'), (1, 'h'), (1, 's')]) == 0

# Test case 3
assert my_function(1, [(1, 'm'), (25, 's'), (1, 'h'), (1, 's')]) == -1

# Test case 4
assert my_function(1, []) == -1
```

### Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 56: Converting Time 2

### Exercise Description

Write a function `my_function(K, L)` that is given an integer number `K` and a list of tuples `L`. Each tuple is a triple consisting of three integer numbers, corresponding to **hours, minutes, and seconds**.

After computing the total number of seconds given by each tuple, the function should return the position `i` of the tuple in list `L` such that the sum of seconds up to position `i` is **smaller than or equal to** `K`. Positions start from 0.

If the input list is empty or the number of seconds specified by the tuple in position 0 is larger than `K`, the function should return `-1`.

Example: `my_function(4000, [(0, 2, 3), (1, 3, 5), (2, 0, 32)])` should return 1.

Your solution

```python
def my_function(K, L):
    L_sec = []
    for h, m, s in L:
        L_sec.append(s + m * 60 + h * 3600)

    if len(L) == 0 or L_sec[0] > K:
        return -1

    cumsum = [0 for _ in range(len(L))]
    for i in range(len(L)):
        cumsum[i] = sum(L_sec[: i + 1])

    TF = [item <= K for item in cumsum]

    if False in TF:
        return TF.index(False) - 1
    else:
        return len(L) - 1
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(4000, [(0, 2, 3), (1, 3, 5), (2, 0, 32)]) == 1

# Test case 2
assert my_function(344000, [(0, 0, 3), (0, 0, 5), (2, 0, 32)]) == 2

# Test case 3
```

```python
assert my_function(2, [(0, 0, 3), (1, 0, 5), (2, 0, 32)]) == -1

# Test case 4
assert my_function(2, []) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 57: Divisible by 4

Exercise Description

You have to compute how many numbers divisible by 4 with remainder 1 or 2 you have to sum up until the value of the sum becomes **larger than or equal** to `n`.

The series of numbers divisible by 4 with remainder 1 or 2 starts with 1, 2, 5, 6, 9, 10, ... (since dividing 1 by 4 gives remainder 1, 2 gives remainder 2, 5 gives remainder 1, and so on).

Write a function `my_function(n)` that gets an integer value `n` as input and returns the **number of terms** in the smallest sum whose value is ≥ `n`. The sum of 0 terms is equal to 0.

Example: if the input is 25, the output is 6 because 1 + 2 + 5 + 6 + 9 = 23 < 25 but 1 + 2 + 5 + 6 + 9 + 10 = 33 ≥ 25.

Your solution

```
def my_function(n):
    x = 1
    mySum = 0
    i = 0
    while mySum < n:
        if x % 4 == 1 or x % 4 == 2:
            mySum = mySum + x
            i = i + 1
        x = x + 1
    return i
```

Test your solution

```
# Test case 1 (Example)
assert my_function(25) == 6

# Test case 2
assert my_function(8) == 3

# Test case 3
assert my_function(75) == 9
```

```python
# Test case 4
assert my_function(0) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 58: Fitting sum

Exercise Description

You are given an integer number `n` as input, representing the maximum amount available. Sum all the natural numbers (1, 2, 3, ...) that fit in `n`, i.e., all natural numbers whose cumulative sum is smaller than or equal to `n`.

Define a function `last_fit(n)` that returns the **last number** that fits in this sum.

Example: if `n = 12`, the output is 4 because `1 + 2 + 3 + 4 <= 12` but `1 + 2 + 3 + 4 + 5 > 12`.

Your solution

```
def last_fit(n):
    mySum = 0
    toBeAdded = 0
    while mySum <= n:
        toBeAdded = toBeAdded + 1
        mySum = mySum + toBeAdded
    return toBeAdded - 1
```

Test your solution

```
# Test case 1 (Example)
assert last_fit(12) == 4

# Test case 2
assert last_fit(8) == 3

# Test case 3
assert last_fit(10) == 4

# Test case 4
assert last_fit(100) == 13
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

[🔗 Visualize YOUR solution in Python Tutor]

---

# Exercise 59: KM CM to M

Exercise Description

Write a function `my_function(S, L)` that is given an integer number `S` and a list of tuples `L`. Each tuple is a pair consisting of an integer number and a character `'k'` (kilometers), `'c'` (centimeters), or `'m'` (meters).

After converting kilometers and centimeters to meters, the function should return the position `i` of the tuple in list `L` such that the sum of meters up to position `i` is **smaller than or equal to** `S`. Positions start from 0.

If the input list is empty or the number of meters specified by the tuple in position 0 is larger than `S`, the function should return `-1`.

Example: `my_function(1000, [(2, 'm'), (30000, 'c'), (5, 'k')])` should return 1, because 30000 cm = 300 m, 5 km = 5000 m, and 2 + 300 = 302 ≤ 1000, while 2 + 300 + 5000 = 5302 > 1000.

Your solution

```python
def my_function(S, L):
    if not L or L[0][0] > S:
        return -1

    total = 0
    i = 0
    while i < len(L):
        value, unit = L[i]
        if unit == 'k':
            value *= 1000
        elif unit == 'c':
            value *= 0.01

        total += value
        if total > S:
            return i - 1
        i += 1
    return i - 1
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(1000, [(2, 'm'), (30000, 'c'), (5, 'k')]) == 1

# Test case 2
assert my_function(1000, [(2, 'm'), (30000, 'c'), (5, 'c')]) == 2

# Test case 3
assert my_function(1, [(20, 'k'), (30000, 'c'), (5, 'k')]) == -1
```

```python
# Test case 4
assert my_function(1000, []) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

[🔗 Visualize YOUR solution in Python Tutor]

---

# Exercise 60: Larger in a sequence

Exercise Description

Write a function `first_larger(a, b, n)` that receives three natural numbers `a`, `b`, and `n`.

Starting from `a` and `b`, the function generates a sequence where each element is equal to the sum of the previous two elements. It then returns the **first element in the sequence that is strictly larger than** `n`.

Example: if `a = 1`, `b = 2`, and `n = 50`, the sequence is 1, 2, 3, 5, 8, 13, 21, 34, 55, ... and the function should return 55.

Your solution

```python
def first_larger(a, b, n):
    second_to_last = a
    last = b

    while last < n:
        new = last + second_to_last
        second_to_last = last
        last = new

    return last
```

Test your solution

```python
# Test case 1 (Example)
assert first_larger(1, 2, 50) == 55

# Test case 2
assert first_larger(1, 2, 15) == 21

# Test case 3
a = 2; b = 3; n = 8
assert first_larger(a, b, n) == 8

# Test case 4
a = 1; b = 1; n = 1000
assert first_larger(a, b, n) == 1597
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 61: Sum 2 Series

## Exercise Description

You have to compute the sum of a series of numbers 2, 22, 222, 2222, and so on, until the value of the sum becomes larger than or equal to `n`.

Write a function `my_function(n)` that gets an integer value `n` as input parameter and returns the **number of terms** in the smallest sum whose value is ≥ `n`. The sum of 0 terms is equal to 0.

Hint: each number can be obtained from the previous one in the series by multiplying it by 10 and adding 2 (e.g., 2222 = 222 * 10 + 2).

Example: if the input is 25, the output is 3 because 2 + 22 = 24 < 25 but 2 + 22 + 222 = 246 ≥ 25.

Your solution

```python
def my_function(n):
    twos = 2
    mySum = 0
    i = 1
    while mySum < n:
        mySum = mySum + twos
        twos = 10 * twos + 2
        i = i + 1
    return i - 1
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(25) == 3

# Test case 2
assert my_function(1000) == 4

# Test case 3
assert my_function(2468) == 4

# Test case 4
assert my_function(1) == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 62: Sum Fermat

Exercise Description

You have to compute the sum of numbers 11, 101, 1001, 10001, 100001, and so on, until the value of the sum becomes larger than or equal to `n`.

Write a function `my_function(n)` that gets an integer value `n` as input parameter and returns the **number of terms** in the smallest sum whose value is ≥ `n`. The sum of 0 terms is equal to 0.

Hint: each number in the series is a power of 10 plus 1 (e.g., 10001 = 10^4 + 1).

Example: if the input is 250, then the output is 3 because 11 + 101 = 112 < 250 but 11 + 101 + 1001 = 1113 ≥ 250.

Your solution

```python
def my_function(n):
    howMany0 = 0
    mySum = 0
    while mySum < n:
        mySum = mySum + int('1' + howMany0 * '0' + '1')
        howMany0 = howMany0 + 1
    return howMany0
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(250) == 3

# Test case 2
assert my_function(11114) == 4

# Test case 3
assert my_function(3000) == 4

# Test case 4
assert my_function(1) == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 63: Sum Km and Cm to M

Exercise Description

Write a function `my_function(S, L)` that is given an integer number `S` and a list of tuples `L`. Each tuple is a pair consisting of an integer number and a character `'k'` (kilometers), `'c'` (centimeters), or `'m'` (meters).

After converting kilometers and centimeters to meters, the function should return the **largest sum** `s` of meters such that `s <= S`. The sum must be computed starting from position 0 and using elements in consecutive positions in the list.

If the input list is empty or the number of meters specified by the tuple in position 0 is larger than `S`, the function should return `-1`.

Example: `my_function(1000, [(2, 'm'), (30000, 'c'), (5, 'k')])` should return 302, because 30000 cm = 300 m, 5 km = 5000 m, and 2 + 300 = 302 ≤ 1000, while 2 + 300 + 5000 = 5302 > 1000.

Your solution

```python
def my_function(S, L):
    # check if the list is empty
    if not L:
        return -1
    # check if the first element is greater than S
    if L[0][1] == 'k' and L[0][0] * 1000 > S:
        return -1
    if L[0][1] == 'c' and L[0][0] / 100 > S:
        return -1
    if L[0][1] == 'm' and L[0][0] > S:
        return -1

    sum_meters = 0
    max_sum = 0

    for distance, unit in L:
        if unit == 'k':
            distance *= 1000
        elif unit == 'c':
            distance /= 100

        sum_meters += distance

        if sum_meters > S:
            break

        max_sum = sum_meters

    return max_sum
```

Test your solution

```
# Test case 1 (Example)
assert int(my_function(1000, [(2, 'm'), (30000, 'c'), (5, 'k')])) == 302

# Test case 2
assert int(my_function(1000, [(2, 'm'), (3, 'm'), (5, 'm')])) == 10

# Test case 3
assert int(my_function(1, [(2, 'm'), (30000, 'c'), (5, 'k')])) == -1

# Test case 4
assert int(my_function(1000, [])) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 64: Sum of Triangular

## Exercise Description

You have to compute the sum of triangular numbers 1, 3, 6, 10, 15, 21, ... until the value of the sum becomes larger than or equal to `n`.

Write a function `my_function(n)` that gets an integer value `n` as input parameter and returns the **number of terms** in the smallest sum whose value is ≥ `n`. The sum of 0 terms is equal to 0.

Hint: the first triangular number in the sequence (position 1) is 1. For `x > 1`, the triangular number at position `x` is equal to the triangular number at position `x-1` plus `x`.

Example: if the input is 25, then the output is 5 because 1 + 3 + 6 + 10 = 20 < 25 but 1 + 3 + 6 + 10 + 15 = 35 ≥ 25.

## Your solution

```python
def my_function(n):
    i = 1
    last_x = 0
    x = 0
    mySum = 0
    while mySum < n:
        x = last_x + i
        last_x = x
        i = i + 1
        mySum = mySum + x
    return i - 1
```

Test your solution

```
# Test case 1 (Example)
assert my_function(25) == 5

# Test case 2
assert my_function(56) == 6

# Test case 3
assert my_function(60) == 7

# Test case 4
assert my_function(1) == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 65: Classroom Attendance Tracker

Exercise Description

Imagine you're helping a school modernize its attendance system. Each classroom has a unique ID and a roster of students. The school wants to track attendance daily, marking students as present or absent.

Write a function `update_attendance(current_attendance, attendance_updates)` that:

- `current_attendance` is a dictionary where keys are classroom IDs and values are dictionaries mapping student names to their attendance status (`'Present'` or `'Absent'`).
- `attendance_updates` is a list of tuples `(class_id, student_name, status)` with the updated status.

The function should apply all updates and return the updated dictionary. If a classroom or student does not exist yet, add them.

Your solution

```
def update_attendance(current_attendance, attendance_updates):
    for t in attendance_updates:
        # unpack the tuple
        classID, studentName, attendanceStatus = t
        # if the class ID exists in the dictionary, update the attendance status for
        if classID in current_attendance:
            current_attendance[classID][studentName] = attendanceStatus
        # otherwise, initialize a new class ID to the dictionary and add the student
```

```
        else:
            current_attendance[classID] = {studentName: attendanceStatus}
    # job done
    return current_attendance
```

Test your solution

```
# Test case 1 (Example)
current_attendance = {
    'Class 1A': {'Alice': 'Present', 'Bob': 'Absent'},
    'Class 2B': {'Charlie': 'Present', 'Daisy': 'Present'},
}

attendance_updates = [
    ('Class 1A', 'Alice', 'Absent'),
    ('Class 2B', 'Eva', 'Present'),
    ('Class 3C', 'Frank', 'Present'),
]

updated_attendance = update_attendance(current_attendance, attendance_updates)
assert updated_attendance == {
    'Class 1A': {'Alice': 'Absent', 'Bob': 'Absent'},
    'Class 2B': {'Charlie': 'Present', 'Daisy': 'Present', 'Eva': 'Present'},
    'Class 3C': {'Frank': 'Present'},
}

# Test case 2
current_attendance = {
    'Class 3A': {'John': 'Present', 'Sarah': 'Absent'},
    'Class 4B': {'Mike': 'Absent', 'Emily': 'Present'},
}
```

```python
attendance_updates = [
    ('Class 3A', 'Sarah', 'Present'),
    ('Class 4B', 'Mike', 'Present'),
    ('Class 5C', 'Anna', 'Present'),
]

updated_attendance = update_attendance(current_attendance, attendance_updates)
assert updated_attendance == {
    'Class 3A': {'John': 'Present', 'Sarah': 'Present'},
    'Class 4B': {'Mike': 'Present', 'Emily': 'Present'},
    'Class 5C': {'Anna': 'Present'},
}

# Additional tests adapted from the import notebook follow the same pattern
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 66: Employee Skillset Tracker

## Exercise Description

You are assisting the HR department of TechGiant Inc. to manage employees' skillsets.

Write a function `update_employee_skills(current_skills, skill_updates)` that:

- `current_skills` is a dictionary where keys are employee IDs and values are lists of skills.
- `skill_updates` is a list of tuples `(employee_id, list_of_skills, action)` where `action` is either `'add'` or `'remove'`.

The function should:

- If `action == 'add'`, **add** all skills in the list to that employee's skills (without removing existing ones).
- If `action == 'remove'`, **remove** each listed skill from the employee's skills (if present).
- If an employee ID is not present in `current_skills`, add it.
- If removing skills leaves an employee with no skills, remove that employee from the dictionary.

The function must return the updated dictionary.

## Your solution

```python
def update_employee_skills(current_skills, skill_updates):
    for t in skill_updates:
        # unpack the tuple
        employeeID, listOfSkills, action = t
        # if we already have the employee in the dictionary ...
        if employeeID in current_skills.keys():
            if action == 'add':
                current_skills[employeeID].extend(listOfSkills)
            elif action == 'remove':
                for skill in listOfSkills:
                    if skill in current_skills[employeeID]:
                        current_skills[employeeID].remove(skill)
        # otherwise we just add it
        else:
            current_skills[employeeID] = listOfSkills

    # let's get rid of employees with no skills
    for employeeID in list(current_skills.keys()):
        if len(current_skills[employeeID]) == 0:
            del current_skills[employeeID]

    return current_skills
```

Test your solution

```python
# Test case 1 (Example)
current_skills = {
    101: ['Python', 'SQL'],
    102: ['Java', 'C++', 'Python'],
    103: ['HTML', 'CSS'],
```

```
}

skill_updates = [
    (101, ['Java'], 'add'),
    (104, ['Python', 'JavaScript'], 'add'),
    (102, ['C++'], 'remove'),
    (103, ['HTML'], 'remove'),
]

updated_skills = update_employee_skills(current_skills, skill_updates)
assert updated_skills == {
    101: ['Python', 'SQL', 'Java'],
    102: ['Java', 'Python'],
    103: ['CSS'],
    104: ['Python', 'JavaScript'],
}

# Additional tests follow the same structure as in the import notebook
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 67: Inventory Management in Gridmart

Exercise Description

Gridmart is organized as a grid where each cell stores the quantity of a product category.

Write a function `update_inventory(current_inventory, updates)` that:

- `current_inventory` is a 2D list (list of lists) where each sublist is a row and each element is the quantity of a product category.
- `updates` is a list of tuples `(row, column, delta)` describing how many products to add (positive) or subtract (negative).

The function should:

- For each update, check if `(row, column)` is inside the grid. If any update refers to an invalid position, return `None`.
- Otherwise, apply all updates in order, modifying `current_inventory`.
- If after an update any quantity becomes negative, return `-1`.
- If all updates are valid and no quantity goes negative, return the updated 2D list.

Your solution

```
def update_inventory(current_inventory, updates):
    # calculate the size of the grid
    nRows = len(current_inventory)
```

```
    nCols = len(current_inventory[0])
    # run it through the updates
    for t in updates:
        # unpack the tuple
        row, column, quantity = t
        # check if the row number is valid: if not, return None
        if row < 0 or row >= nRows:
            return None
        # check if the column number is valid: if not, return None
        if column < 0 or column >= nCols:
            return None
        # if no returns are triggered, this means that the (row, column) pair is vali
        current_inventory[row][column] += quantity
        # do we have a negative number? if so, return -1
        if current_inventory[row][column] < 0:
            return -1
    # job done
    return current_inventory
```

Test your solution

```
# Test case 1 (Example)
current_inventory = [[20, 15, 10], [30, 25, 5], [12, 8, 3]]
updates = [(0, 1, -5), (1, 0, 10), (2, 2, -2)]
assert update_inventory(current_inventory, updates) == [[20, 10, 10], [40, 25, 5], [1

# Test case 2: invalid column index
current_inventory = [[40, 30, 20], [25, 35, 45], [15, 10, 5]]
updates = [(0, 2, 5), (1, 3, -15), (2, 0, 10)]
assert update_inventory(current_inventory, updates) is None
```

```
# Test case 3: negative quantity after update
current_inventory = [[50, 60], [20, 30], [70, 80]]
updates = [(0, 0, -100), (1, 1, 20), (2, 0, -15)]
assert update_inventory(current_inventory, updates) == -1

# Test case 4
current_inventory = [[100, 200, 300], [400, 500, 600]]
updates = [(0, 1, -50), (1, 2, 25), (1, 0, -40)]
assert update_inventory(current_inventory, updates) == [[100, 150, 300], [360, 500, 6

# Test case 5
current_inventory = [[5, 10, 15], [20, 25, 30], [35, 40, 45], [50, 55, 60]]
updates = [(3, 2, -10), (2, 1, 20), (0, 0, 5)]
assert update_inventory(current_inventory, updates) == [[10, 10, 15], [20, 25, 30], [
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 68: Library Book Tracker

Exercise Description

A library wants to track whether books are currently 'in' or 'out'.

Write a function `update_library(current_status, transactions)` that:

- `current_status` is a dictionary mapping book titles to their status (`'in'` or `'out'`).
- `transactions` is a list of tuples `(title, status)` where `status` is `'in'` or `'out'`.

The function should apply each transaction in order, updating or inserting entries in the dictionary, and then return the updated dictionary. If a title does not exist yet, add it.

Your solution

```
def update_library(current_status, transactions):
    for transaction in transactions:
        book, status = transaction
        current_status[book] = status
    return current_status
```

Test your solution

```
# Test case 1 (Example)
current_status = {
    'The Great Gatsby': 'in',
    'To Kill a Mockingbird': 'out',
    '1984': 'in',
}
```

```python
transactions = [
    ('To Kill a Mockingbird', 'in'),
    ('The Catcher in the Rye', 'out'),
    ('1984', 'out'),
]

updated_status = update_library(current_status, transactions)
assert updated_status == {
    'The Great Gatsby': 'in',
    'To Kill a Mockingbird': 'in',
    '1984': 'out',
    'The Catcher in the Rye': 'out',
}

# Additional tests mirror the import notebook cases
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 69: The Classroom Gradebook

Exercise Description

Mr. Alan maintains a gradebook where each student's test scores are stored in a row.

Write a function `calculate_student_averages(gradebook)` that:

- Receives a 2D list `gradebook`, where each inner list contains the scores of one student.
- Returns a list of tuples `(student_index, average_score)` where `student_index` is the index of the student in `gradebook` and `average_score` is their average rounded to 2 decimal places.

Caveats:

- Students with no scores are represented by an empty inner list: their average is defined to be 0.

Your solution

```
def calculate_student_averages(gradebook):
    output = []
    for i in range(len(gradebook)):
        sumOfGrades = sum(gradebook[i])
        numOfGrades = len(gradebook[i])
        if numOfGrades == 0:
            output.append((i, 0))
```

```
        else:
            output.append((i, round(sumOfGrades / numOfGrades, 2)))
    return output
```

Test your solution

```
# Test case 1 (Example)
gradebook = [[78, 82, 85], [92, 88, 91], [83, 76, 79], [95, 92]]
assert calculate_student_averages(gradebook) == [(0, 81.67), (1, 90.33), (2, 79.33),

# Further tests follow the patterns in the import notebook
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 70: The Seating Arrangement Puzzle

Exercise Description

The community hall seating is organized as a 2D grid of 'Occupied' / 'Available' seats.

Write a function `update_seating_arrangement(current_seating, updates)` that:

- `current_seating` is a 2D list where each inner list is a row of seats.
- `updates` is a list of tuples `(row, col, new_status)` where `new_status` is `'Occupied'` or `'Available'`.

The function should:

- First check that the grid is regular (all rows have the same length). If not, return `-1`.
- For each update, if `(row, col)` is inside the grid, update that seat's status. If `(row, col)` is outside the grid, ignore that update.
- Return the updated 2D list.

Your solution

```
def update_seating_arrangement(current_seating, updates):
    # calculate the number of rows
    n = len(current_seating)
    # calculate the number of columns for each row
    m = []
    for i in range(n):
        m.append(len(current_seating[i]))
    # do we have an irregular grid?
    if len(set(m)) > 1:
        return -1
```

```python
    else:
        m = m[0]
    # update the grid
    for update in updates:
        # unpack the tuple
        row, col, new_value = update
        # do row and col fit in the arrangement?
        if not (row < 0 or row >= n or col < 0 or col >= m):
            current_seating[row][col] = new_value
    # job done
    return current_seating
```

Test your solution

```python
# Test case 1 (Example)
current_seating = [
    ['Available', 'Occupied', 'Available'],
    ['Occupied', 'Occupied', 'Available'],
    ['Available', 'Available', 'Available'],
]

updates = [(0, 0, 'Occupied'), (1, 2, 'Occupied'), (2, 1, 'Occupied')]
assert update_seating_arrangement(current_seating, updates) == [
    ['Occupied', 'Occupied', 'Available'],
    ['Occupied', 'Occupied', 'Occupied'],
    ['Available', 'Occupied', 'Available'],
]

# Additional tests mirror the import notebook behaviour
```

### Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 71: Count Pairs

### Exercise Description

Write a function `my_function(n, a, b, k)` that returns the **number of pairs** `(x, y)` such that:

1. `x` is a multiple of 3, strictly larger than 0 and smaller than or equal to `n`.
2. `y` is a number strictly larger than `a` and strictly smaller than `b`.
3. The difference between `x` and `y` is at most `k` (included), i.e. `x - y <= k`.

### Your solution

```
def my_function(n, a, b, k):
    numPairs = 0
```

```python
    for x in range(3, n + 1, 3):
        for y in range(a + 1, b):
            if x - y <= k:
                numPairs = numPairs + 1
    return numPairs
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(32, 9, 21, 12) == 95

# Test case 2
assert my_function(32, 3, 21, 12) == 130

# Test case 3
assert my_function(32, 3, 21, 3) == 79

# Test case 4
assert my_function(2, 3, 21, 3) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

# Exercise 72: Count Pairs Between

Exercise Description

Write a function `my_function(n, a, b, k)` that returns the **number of pairs** `(x, y)` such that:

1. `y` is an even number strictly larger than 0 and smaller than or equal to `n`.
2. `x` is a number strictly larger than `a` and strictly smaller than `b`.
3. The product `x * y` is at most `k` (included).

Your solution

```
def my_function(n, a, b, k):
    numPairs = 0
    for y in range(2, n + 1, 2):
        for x in range(a + 1, b):
            if x * y <= k:
                numPairs = numPairs + 1
    return numPairs
```

Test your solution

```
# Test case 1 (Example)
assert my_function(2, 1, 6, 10) == 4
```

```python
# Test case 2
assert my_function(2, 10, 60, 100) == 40

# Test case 3
assert my_function(6, 2, 6, 100) == 9

# Test case 4
assert my_function(6, 2, 6, 1) == 0
```

### Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 73: Maxsum Pairs

### Exercise Description

Write a function `my_function(n, a, b, k)` that returns the **maximum sum** `x + y` taken among all pairs `(x, y)` such that:

- `y` is a multiple of 3, strictly larger than 0 and strictly smaller than `n`.
- `x` is a number strictly larger than `a` and strictly smaller than `b`.
- The difference between `x` and `y` is at least `k` (included), i.e. `x - y >= k`.

If no pair satisfies the conditions, the function should return 0.

Your solution

```python
def my_function(n, a, b, k):
    bestSum = 0
    for y in range(3, n, 3):
        for x in range(a + 1, b):
            if x - y >= k:
                if x + y > bestSum:
                    bestSum = x + y
    return bestSum
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(20, 1, 21, 12) == 26

# Test case 2
assert my_function(3, 1, 21, 12) == 0

# Test case 3
assert my_function(20, 1, 210, 12) == 227
```

```python
# Test case 4
assert my_function(20, 1, 6, 2) == 8
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 74: Maxsum pairs Odd

Exercise Description

Write a function `my_function(n, a, b, k)` that returns the **maximum sum** `x + y` taken among all pairs `(x, y)` such that:

1. `x` is a positive odd number smaller than or equal to `n`.
2. `y` is a number strictly larger than `a` and smaller than or equal to `b`.
3. The product `x * y` is at most `k` (included).

If no pair satisfies the conditions, the function should return 0.

Your solution

```python
def my_function(n, a, b, k):
    bestSum = 0
    for x in range(1, n + 1, 2):
        for y in range(a + 1, b + 1):
            if x * y <= k:
                if x + y > bestSum:
                    bestSum = x + y
    return bestSum
```

Test your solution

```python
# Test case 1 (Example)
assert my_function(20, 1, 6, 2) == 3

# Test case 2
assert my_function(2, 1, 6, 2) == 3

# Test case 3
assert my_function(2, 1, 6, 20) == 7

# Test case 4
assert my_function(2, 1, 6, 1) == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code**

**before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 75: Difference between main and anti-diagonal

Exercise Description

Let `M` be an `n × m` matrix of numbers organized as a list-of-lists with `n` lists where each inner list has length `m`. You can assume that all inner lists have the same size `m` and you can assume `M` to be non-empty.

Write a Python function called `my_function(M)` that, given a matrix `M`, returns the absolute value of the difference between the sum of its main diagonal and the sum of its anti-diagonal.

Keep in mind the following caveats and definitions:

- normally, main diagonal and anti-diagonal are defined for square matrices only (namely matrices whose number of rows is equal to the number of columns): if `n` is not equal to `m`, the function must return `-1`
- the main diagonal of a matrix is defined as the list of entries lying from the top left to the bottom right corner of the matrix
- the anti-diagonal of a matrix is defined as the list of entries lying from the top right to the bottom left corner of the matrix

Your solution

```
# Teacher solution
def my_function(M):
    n = len(M)
    m = len(M[0])

    # is the matrix a square matrix?
    if n != m:
        return -1

    sum_diag = 0
    for i in range(n):
        sum_diag += M[i][i]

    sum_antidiag = 0
    for i in range(n):
        sum_antidiag += M[i][n-1-i]

    if sum_antidiag > sum_diag:
```

```python
            return sum_antidiag - sum_diag
        elif sum_antidiag < sum_diag:
            return sum_diag - sum_antidiag
        else:
            return 0
```

Test your solution

```python
# Test case 1 (Example)
M = [[1,2,3], [4,5,6], [7,8,9]]
assert my_function(M) == 0

# Additional tests
M = [[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]]
assert my_function(M) == -1

M = [[29, 10, 12, 33, 83, 28, 32, 81, 91, 84],
 [98, 60, 84, 68, 60, 18, 65, 10, 51, 50],
 [20, 15, 33, 33, 64, 37, 82, 50, 90, 45],
 [80, 38, 9, 43, 69, 82, 78, 39, 46, 44],
 [38, 69, 68, 20, 62, 53, 18, 59, 12, 12],
 [52, 14, 48, 3, 72, 64, 74, 45, 28, 97],
 [6, 27, 4, 2, 82, 67, 79, 96, 50, 55],
 [23, 83, 46, 45, 17, 26, 39, 48, 9, 75],
 [7, 58, 84, 63, 98, 25, 50, 78, 35, 30],
 [55, 36, 22, 32, 45, 14, 14, 5, 75, 34]]
assert my_function(M) == 62
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 76: Replace odd columns with previous max

Exercise Description

Let `M` be an n x m matrix of numbers organized as a list-of-lists with n lists where each inner list has length m. You can assume that all inner lists have the same size m and you can assume `M` to have at least 2 columns.

Write a Python function called `my_function(M)` that, given a matrix `M`, returns a modified version of `M` where the values in the odd columns are replaced with the maximum element of the previous even column.

Furthermore:

- if `n` is not equal to `m`, the function must return `-1`
- if `n` is equal to `m`, but there is at least one negative element (regardless of its position), the function must return `-2`

Your solution

```python
# Teacher solution
def my_function(M):
    n = len(M)
    m = len(M[0])

    # is the matrix a square matrix?
    if n != m:
        return -1

    # any negative elements?
    for i in range(n):
        for j in range(m):
            if M[i][j] < 0:
                return -2
    maxval = 0
    for j in range(m):
        if j % 2 == 0:
            l = []
            for i in range(n):
                l.append(M[i][j])
            maxval = max(l)
        else:
            for i in range(n):
                M[i][j] = maxval
    return M
```

Test your solution

```
# Test case 1 (Example)
M = [[1,2,3], [4,5,6], [7,8,9]]
assert my_function(M) == [[1, 7, 3], [4, 7, 6], [7, 7, 9]]
```

Test your solution

```
# Additional tests
M = [[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]]
assert my_function(M) == -1

M = [[1,2,3], [-0.01,5,6], [7,8,9]]
assert my_function(M) == -2

M = [[1, 2, 3, 0, 4], [3, 3, 1, 0, 3], [7, 3, 8, 4, 0], [1, 6, 7, 9, 0], [2, 4, 6, 9,
assert my_function(M) == [[1, 7, 3, 8, 4], [3, 7, 1, 8, 3], [7, 7, 8, 8, 0], [1, 7, 7
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 77: Replace column i with row i

Exercise Description

Write a function `my_function(M, i)` that is given a list of lists of numbers `M` and an integer number `i` such that:

- `M` represents a square matrix containing `D x D` elements: you can assume that all its sublists (corresponding to rows) have the same length and that the number `D` of rows equals the number `D` of columns.
- Integer `i` is a row and column index: it is larger than or equal to `0`, and strictly smaller than `D`.

The function should modify `M` by replacing the `i`-th column with the `i`-th row and should then return the modified matrix.

Back-of-the-envelope example: if `M = [[1,2,3],[4,5,6],[7,8,9]]` and `i = 1`, the returned matrix should be `[[1,4,3],[4,5,6],[7,6,9]]`.

Your solution

```
# Teacher solution
def my_function(M, i):
    D = len(M)
    M_new = M.copy()
    # get row i
    tmp = M[i]
```

```
    # replace column i
    for k in range(D):
        M_new[k][i] = tmp[k]
    # done
    return M_new
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1,2,3],[4,5,6],[7,8,9]],1)
assert a == [[1, 4, 3], [4, 5, 6], [7, 6, 9]]
```

Test your solution

```
# Additional tests
a = my_function([[1,2,3],[4,5,6],[7,8,9]],2)
assert a == [[1, 2, 7], [4, 5, 8], [7, 8, 9]]

a = my_function([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]],0)
assert a == [[1, 2, 3, 4], [2, 6, 7, 8], [3, 10, 11, 12], [4, 14, 15, 16]]

a = my_function([[1, 2], [3, 4]],1)
assert a == [[1, 3], [3, 4]]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

[🔗 Visualize YOUR solution in Python Tutor]

---

# Exercise 78: Replace row i with column j

Exercise Description

Write a function `my_function(M, i, j)` that is given a list of lists `M` of numbers and two integer numbers `i` and `j` such that:

- `M` represents a square matrix containing `D x D` elements: you can assume that all its sublists (corresponding to rows) have the same length and that the number `D` of rows equals the number `D` of columns.
- Integer `i` is a row index: it is larger than or equal to `0`, and strictly smaller than `D`.
- Integer `j` is a column index: it is larger than or equal to `0`, and strictly smaller than `D`.

The function should modify `M` by replacing the `i`-th row with the `j`-th column and should then return the modified matrix.

Back-of-the-envelope example: if `M = [[1,2,3],[4,5,6],[7,8,9]]` and `i = 1`, `j = 2`, the returned matrix should be `[[1,2,3],[3,6,9],[7,8,9]]`.

Your solution

```python
# Teacher solution
def my_function(M, i, j):
    D = len(M)
    M_new = M.copy()
    # get column j
    tmp = []
    for k in range(D):
        tmp.append(M[k][j])
    # replace
    M_new[i] = tmp
    # done
    return M_new
```

Test your solution

```python
# Test case 1 (Example)
a = my_function([[1,2,3],[4,5,6],[7,8,9]],1,2)
assert a == [[1, 2, 3], [3, 6, 9], [7, 8, 9]]
```

Test your solution

```python
# Additional tests
a = my_function([[1,2,3],[4,5,6],[7,8,9]],2,2)
assert a == [[1, 2, 3], [4, 5, 6], [3, 6, 9]]

a = my_function([[1,2],[4,5]],0,1)
assert a == [[2, 5], [4, 5]]
```

```python
a = my_function([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [13, 14, 15, 16]],3,0)
assert a == [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [1, 5, 9, 13]]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 79: Replace row i with column i

Exercise Description

Write a function `my_function(M, i)` that is given a list `M` of lists of numbers and an integer number `i` such that:

- `M` represents a square matrix containing `D x D` elements: you can assume that all its sublists (corresponding to rows) have the same length and that the number `D` of rows equals the number `D` of columns.
- Integer `i` is a row and column index: it is larger than or equal to `0`, and strictly smaller than `D`.

The function should modify `M` by replacing the `i`-th row with the `i`-th column and should then return the modified matrix.

Back-of-the-envelope example: if `M = [[1,2,3],[4,5,6],[7,8,9]]` and `i = 2`, the returned modified matrix should be `[[1,2,3],[4,5,6],[3,6,9]]`.

Your solution

```
# Teacher solution
def my_function(M, i):
    M_new = M.copy()
    col_i = [row[i] for row in M]
    M_new[i] = col_i
    return M_new
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1,2,3],[4,5,6],[7,8,9]],2)
assert a == [[1, 2, 3], [4, 5, 6], [3, 6, 9]]
```

Test your solution

```
# Additional tests
a = my_function([[1,2,3,4],[5,6,7,8],[9,10,11,12],[13,14,15,16]],3)
assert a == [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [4, 8, 12, 16]]

a = my_function([[1, 2], [3, 4]],0)
assert a == [[1, 3], [3, 4]]
```

```
a = my_function([[1, 2], [3, 4]],1)
assert a == [[1, 2], [2, 4]]
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 80: Sum of i-th row and i-th column

Exercise Description

Write a function `my_function(M, i)` that is given a list of lists of numbers `M` and an integer number `i` such that:

- `M` represents a square matrix containing `D x D` elements: you can assume that all of its sublists have the same length and that the number `D` of rows equals the number `D` of columns.
- Integer `i` is a row and column index: it is larger than or equal to `0`, and strictly smaller than `D`.

The function should return the sum of the elements in the `i`-th row and the elements in the `i`-th column. Element in row `i` and column `i` should be counted twice.

Back-of-the-envelope example: if `M = [[1, 2, 3], [4, 5, 8], [7, 8, 0]]` and `i = 2`, the code should return `26` because `7 + 8 + 0 + 3 + 8 + 0 = 26`.

Your solution

```
# Teacher solution
def my_function(M, i):
    row_i = M[i]
    col_i = [row[i] for row in M]
    return sum(row_i) + sum(col_i)
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1, 2, 3],[4, 5, 8],[7, 8, 0]], 2)
assert a == 26
```

Test your solution

```
# Additional tests
a = my_function([[1, 2, 3], [4, 5, 8], [1, 8, 0]], 0)
assert a == 12

a = my_function([[1, 2],[4, 5]], 1)
assert a == 16

a = my_function([[1, 2, 3, 0], [4, 5, 8, 0], [0, 8, 0, 0], [0, 8, 0, 0]], 0)
assert a == 11
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 81: Sum of items in column range

Exercise Description

Write a function `my_function(L, i, j)` that is given a list `L` of lists of numbers and two integer numbers `i` and `j`. `L` represents a matrix with `R` rows and `C` columns (`R` and `C` are not given as input). You can assume that all the `R` sublists have the same length `C`, with `0 <= i < C` and `0 <= j < C`.

The function shall return the sum of items in columns between column `i` and column `j`, or `0` if the matrix is empty or `i > j`.

Back-of-the-envelope example: `my_function([[1,2,3,4],[10,20,30,40],[5,6,7,8]],1,2)` should return 68 (2+20+6+3+30+7).

Your solution

```python
# Teacher solution
def my_function(L, i, j):
    if i > j or L == []:
        return 0

    R = len(L)

    s = 0
    for r in range(R):
        for c in range(i, j + 1):
            s = s + L[r][c]
    return s
```

Test your solution

```python
# Test case 1 (Example)
a = my_function([[1,2,3,4],[10,20,30,40],[5,6,7,8]],1,2)
assert a == 68
```

Test your solution

```
# Additional tests
a = my_function([[1,2,3,4],[10,20,30,40],[5,6,7,8]],0,1)
assert a == 44

a = my_function([[1,2,3,4],[1,2,3,4],[5,6,7,8]],0,1)
assert a == 17

a = my_function([[1,2,3,4],[1,2,3,4],[5,6,7,8],[10,20,30,50]],0,2)
assert a == 90
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 82: Sum of items in row range

Exercise Description

Write a function `my_function(L, i, j)` that is given a list of lists of numbers `L` and two integer numbers `i` and `j`. `L` represents a matrix with `R` rows and `C` columns (`R` and `C` are not given as input). You can assume that all the `R` sublists have the same length `C`, with `0 <= i < R` and `0 <= j < R`.

The function shall return the sum of items in rows between row `i` and row `j`, or `0` if the matrix is empty or `i > j`.

Back-of-the-envelope example: `my_function([[1,2,3,4],[10,20,30,40],[5,6,7,8]],1,2)` should return 126 (10+20+30+40+5+6+7+8).

Your solution

```
# Teacher solution
def my_function(L, i, j):
    if i > j or L == []:
        return 0

    R = len(L)
    C = len(L[0])

    s = 0
    for c in range(C):
        for r in range(i, j + 1):
            s = s + L[r][c]
    return s
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1,2,3,4],[10,20,30,40],[5,6,7,8]],1,2)
assert a == 126
```

Test your solution

```
# Additional tests
a = my_function([[1,2,3,4],[10,20,30,40],[5,6,7,8]],0,1)
assert a == 110

a = my_function([[1,2,3,4],[1,2,3,4]],0,1)
assert a == 20

a = my_function([[1,2,3,4],[1,2,3,4],[5,0,0,10]],0,2)
assert a == 35
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 83: Sum of main diagonal

Exercise Description

Write a function `my_function(L)` that accepts a list of lists `L` of numbers. `L` represents a square matrix and you can assume that all sublists have the same length and that the number of rows equals the number of columns.

The function shall return the sum of items in the top-left to bottom-right diagonal, or `0` if the matrix is empty.

Back-of-the-envelope example: `my_function([[1,2,3],[1,2,3],[1,2,3]])` returns 6 (1+2+3).

Your solution

```
# Teacher solution
def my_function(L):
    n = len(L)
    if n == 0:
        return 0
    s = 0
    for i in range(n):
        s += L[i][i]
    return s
```

Test your solution

```python
# Test case 1 (Example)
assert my_function([[1,2,3],[1,2,3],[1,2,3]]) == 6
```

Test your solution

```python
# Additional tests
assert my_function([]) == 0
assert my_function([[1,1,1],[1,1,1],[0,0,-2]]) == 0

large_matrix = [
    [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    [11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    [21, 22, 23, 24, 25, 26, 27, 28, 29, 30],
    [31, 32, 33, 34, 35, 36, 37, 38, 39, 40],
    [41, 42, 43, 44, 45, 46, 47, 48, 49, 50],
    [51, 52, 53, 54, 55, 56, 57, 58, 59, 60],
    [61, 62, 63, 64, 65, 66, 67, 68, 69, 70],
    [71, 72, 73, 74, 75, 76, 77, 78, 79, 80],
    [81, 82, 83, 84, 85, 86, 87, 88, 89, 90],
    [91, 92, 93, 94, 95, 96, 97, 98, 99, 100]
]
assert my_function(large_matrix) == 505
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

[🔗 Visualize YOUR solution in Python Tutor]

---

# Exercise 84: Sum of top-right to bottom-left diagonal

Exercise Description

Write a function `my_function(L)` that accepts a list of lists `L` of numbers. `L` represents a square matrix and you can assume that all sublists have the same length and that the number of rows equals the number of columns.

The function shall return the sum of items in the top-right to bottom-left diagonal, or `0` if the matrix is empty.

Back-of-the-envelope example: `my_function([[1,2,3],[1,2,3],[1,2,3]])` returns 6 because 3 + 2 + 1 = 6.

Your solution

```
# Teacher solution
def my_function(L):
    D = len(L)
    s = 0
    for i in range(D):
```

```
        s = s + L[i][-(i + 1)]
    return s
```

Test your solution

```
# Test case 1 (Example)
a = my_function([[1,2,3],[1,2,3],[1,2,3]])
assert a == 6
```

Test your solution

```
# Additional tests
a = my_function([[10,10,3],[1,10,3],[10,2,3]])
assert a == 23

a = my_function([[10, 10, 3, 1],[1, 10, 3, 9],[10, 2, 3, 10], [0, 2, 3, 10]])
assert a == 6

a = my_function([])
assert a == 0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 85: Matrix transpose

Exercise Description

Let `M` be an `n × m` matrix of numbers organized as a list-of-lists with `n` lists where each inner list has length `m`.

Write a function called `my_function(M)` that takes as input a matrix `M` and returns its transpose.

Keep in mind the following caveats and definitions:

- if the input matrix is empty, the function must return `-1`
- the input list-of-lists must indeed be a valid matrix where all rows have the same length `m`: if this does not hold, the function must return `-2`
- the transpose of an `n × m` matrix is a new `m × n` matrix where the item in position `(i, j)` of the original matrix lies in position `(j, i)` in the transposed matrix

Your solution

```
# Teacher solution
def my_function(M):
    # is the matrix empty?
```

```
    if M == []:
        return -1

    n = len(M)
    m = [len(M[i]) for i in range(n)]

    # do all rows have the same length?
    if len(set(m)) != 1:
        return -2
    else:
        m = m[0]

    # create the transpose of matrix M using a double for loop
    MT = []
    for j in range(m):
        row = []
        for i in range(n):
            row.append(M[i][j])
        MT.append(row)
    return MT
```

Test your solution

```
# Test case 1 (Example)
M = [[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]]
assert my_function(M) == [[1, 4, 7, 10], [2, 5, 8, 11], [3, 6, 9, 12]]

# Additional tests
M = [[29, 10, 12, 33, 83, 28, 32, 81, 91, 84],
 [98, 60, 84, 68, 60, 18, 65, 10, 51, 50],
 [20, 15, 33, 33, 64, 37, 82, 50, 90, 45, 8],
```

```python
 [80, 38, 9, 43, 69, 82, 78, 39, 46, 44],
 [38, 69, 68, 20, 62, 53, 18, 59, 12, 12],
 [52, 14, 48, 3, 72, 64, 74, 45, 28, 97],
 [6, 27, 4, 2, 82, 67, 79, 96, 55],
 [23, 83, 46, 45, 17, 26, 39, 48, 9, 75],
 [7, 58, 84, 63, 98, 25, 50, 78, 35, 30],
 [55, 36, 22, 32, 45, 14, 14, 5, 75, 34]]
assert my_function(M) == -2

M = [[29, 10, 12, 33, 83, 28, 32, 81, 91, 84],
 [98, 60, 84, 68, 60, 18, 65, 10, 51, 50],
 [20, 15, 33, 33, 64, 37, 82, 50, 90, 45],
 [80, 38, 9, 43, 69, 82, 78, 39, 46, 44],
 [38, 69, 68, 20, 62, 53, 18, 59, 12, 12],
 [52, 14, 48, 3, 72, 64, 74, 45, 28, 97],
 [6, 27, 4, 2, 82, 67, 79, 96, 50, 55],
 [23, 83, 46, 45, 17, 26, 39, 48, 9, 75],
 [7, 58, 84, 63, 98, 25, 50, 78, 35, 30],
 [55, 36, 22, 32, 45, 14, 14, 5, 75, 34]]
assert my_function(M) == [[29, 98, 20, 80, 38, 52, 6, 23, 7, 55], [10, 60, 15, 38, 69

M = []
assert my_function(M) == -1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

[🔗 Visualize YOUR solution in Python Tutor]

---

# Exercise 86: Max upper diagonal minus min lower diagonal

Exercise Description

Let `M` be an `n × m` matrix of numbers organized as a list-of-lists with `n` lists where each inner list has length `m`. You can assume that all inner lists have the same size `m` and you can assume `M` to be non-empty.

Write a Python function called `my_function(M)` that, given a matrix `M`, returns the difference between the maximum value inside the upper diagonal and the minimum value of the lower diagonal:

- the upper diagonal is defined as the diagonal of the matrix directly above the main diagonal
- the lower diagonal is defined as the diagonal of the matrix directly below the main diagonal
- if `n` is not equal to `m`, the function must return `-1`
- if `n` is equal to `m` but there is at least one negative element (regardless of its position), the function must return `-2`

Your solution

```
# Teacher solution
def my_function(M):
    n = len(M)
    m = len(M[0])

    # is the matrix a square matrix?
    if n != m:
        return -1

    # any negative elements?
    for i in range(n):
        for j in range(m):
            if M[i][j] < 0:
                return -2

    # take the max of the upper diagonal
    s = []
    for i in range(n-1):
```

```
        for j in range(i+1, i+2):
            s.append(M[i][j])
    ma = max(s)
    # take the min of the lower diagonal
    k = []
    for i in range(1, n):
        for j in range(i-1, i):
            k.append(M[i][j])
    mi = min(k)

    return ma - mi
```

Test your solution

```
# Test case 1 (Example)
M = [[1,2,3], [4,5,6], [7,8,9]]
assert my_function(M) == 2
```

Test your solution

```
# Additional tests
M = [[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]]
assert my_function(M) == -1

M = [[1,2,3], [-0.01,5,6], [7,8,9]]
assert my_function(M) == -2

M = [[1, 2, 3, 0, 4], [3, 3, 1, 0, 3], [7, 3, 8, 4, 0], [1, 6, 7, 9, 0], [2, 4, 6, 9,
assert my_function(M) == 1
```

### Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 87: Max upper triangular minus min lower triangular

### Exercise Description

Let `M` be an `n × m` matrix of numbers organized as a list-of-lists with `n` lists where each inner list has length `m`. You can assume that all inner lists have the same size `m` and you can assume `M` to be non-empty.

Write a Python function called `my_function(M)` that, given a matrix `M`, returns the difference between the maximum value of its upper triangular matrix and the minimum value of its lower triangular matrix.

Keep in mind the following caveats and definitions:

- the upper triangular matrix is defined as the portion of the matrix composed by only elements lying above (and including) the main diagonal
- the lower triangular is defined as the portion of the matrix composed by only elements lying below (and excluding in this case) the main diagonal
- normally, triangular matrices are special cases of square matrices (namely matrices whose number of rows is equal to the number of columns): if `n` is not equal to `m`, the function must return `-1`
- if the matrix is a square matrix but there is at least one negative element (regardless of its position), the function must return `-2`

Your solution

```
# Teacher solution
def my_function(M):
    n = len(M)
    m = len(M[0])

    # is the matrix a square matrix?
    if n != m:
        return -1

    # any negative elements?
    for i in range(n):
        for j in range(m):
            if M[i][j] < 0:
                return -2
```

```
    # take the max of the upper triangle
    s = []
    for i in range(n):
        for j in range(i, m):
            s.append(M[i][j])
    ma = max(s)
    # take the min of the lower triangle
    k = []
    for i in range(n):
        for j in range(0, i):
            k.append(M[i][j])
    mi = min(k)
    # peace out!
    return ma - mi
```

Test your solution

```
# Test case 1 (Example)
M = [[1,2,3], [4,5,6], [7,8,9]]
assert my_function(M) == 5
```

Test your solution

```
# Additional tests
M = [[1,2,3], [4,5,6], [7,8,9], [10, 11, 12]]
assert my_function(M) == -1

M = [[1,2,3], [-0.01,5,6], [7,8,9]]
assert my_function(M) == -2
```

```
M = [[1, 2, 3, 0, 4], [3, 3, 1, 0, 3], [7, 3, 8, 4, 0], [1, 6, 7, 9, 0], [2, 4, 6, 9,
assert my_function(M) == 8
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 88: Auction

Exercise Description

Define a class Auction to manage bidding on items. The class shall provide: Constructor: takes item_name (string) and minimum_bid (float) as parameters. place_bid(bidder, amount): adds bid if higher than current highest. Returns True if bid accepted. withdraw_bid(bidder): removes bidder's bid. Returns True if bid existed. get_winner(): returns name of highest bidder (or "No winner" if no bids).

Your solution

```python
class Auction:
    def __init__(self, item_name, minimum_bid):
        self.item_name = item_name
        self.minimum_bid = minimum_bid
        self.best_bid = 0
        self.bids = []

    def place_bid(self, bidder, amount):
        if amount > self.minimum_bid:
            if amount > self.best_bid:
                self.bids.append((bidder, amount))
                self.best_bid = amount

    def withdraw_bid(self, bidder):
        for i in range(len(self.bids)):
            b, a = self.bids[i]
            if b == bidder:
                self.bids.pop(i)
                return True

    def get_winner(self):
        if self.bids == []:
            return "No winner"
        else:
            self.bids = sorted(self.bids, key=lambda x: x[1])
            return self.bids[-1][0]
```

Test your solution

```python
# Test case 1 (Example)
a = Auction("Painting", 100.0)
a.place_bid("John", 150.0)
a.place_bid("Alice", 200.0)
a.place_bid("Bob", 175.0)
winner = a.get_winner()
print(winner)

# Test case 2
a = Auction("Vase", 500.0)
a.place_bid("John", 400.0)
a.place_bid("Alice", 450.0)
winner = a.get_winner()
print(winner)

# Test case 3
a = Auction("Car", 1000.0)
a.place_bid("John", 1500.0)
a.place_bid("Alice", 2000.0)
a.withdraw_bid("Alice")
winner = a.get_winner()
print(winner)

# Test case 4
a = Auction("Watch", 200.0)
winner = a.get_winner()
print(winner)

# Test case 5
a = Auction("Phone", 300.0)
a.place_bid("John", 350.0)
```

```python
a.place_bid("John", 400.0)
a.place_bid("Alice", 375.0)
winner = a.get_winner()
print(winner)
```

```
Alice
No winner
John
No winner
John
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

[🔗 Visualize YOUR solution in Python Tutor]()

---

# Exercise 89: Bank Account

Exercise Description

# BankAccount

Create a Python class called BankAccount that represents a bank account. The class should have the following attributes and methods: Attributes: account_number (string): The account number associated with the bank account. balance (float): The current balance of the bank account. Methods: deposit(amount): Accepts a float amount and adds it to the account's balance. withdraw(amount): Accepts a float amount and subtracts it from the account's balance, if the balance is sufficient. get_balance(): Returns the current balance of the account as a float. get_account_number(): Returns the account number as a string.

Your solution

```
class BankAccount:
    def __init__(self, account_number: str, balance: float = 0.0) -> None:
        self.account_number = account_number
        self.balance = balance

    def deposit(self, amount: float) -> None:
        self.balance += amount

    def withdraw(self, amount: float) -> None:
        if self.balance >= amount:
            self.balance -= amount

    def get_balance(self) -> float:
        return self.balance

    def get_account_number(self) -> str:
        return self.account_number
```

Test your solution

```python
# Test case 1 (Example)
account = BankAccount("12345678", 100.0)
account.deposit(50.0)
print(account.get_balance())

# Test case 2
account = BankAccount("12345678", 100.0)
account.deposit(50.0)
account.withdraw(20.0)
print(account.get_balance())

# Test case 3
account = BankAccount("12345678", 100.0)
print(account.get_account_number())

# Test case 4
account = BankAccount("12345678", 100.0)
account.withdraw(200.0)
print(account.get_balance())
```

```
150.0
130.0
12345678
100.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 90: Barrel

Exercise Description

Define a class Barrel to represent a barrel that contains a liquid. The barrel is initially full of liquid. The class shall provide the following methods. Constructor: takes an input float capacity, that represents the total capacity of the barrel. Initially, the barrel contains an amount of liquid equal to its capacity. draw(bottle_capacity): when called, draw liquid from the barrell to fill (as much as possible) a bottle with capacity bottle_capacity. The method shall return the amount of liquid actually drawn from the barrel.

Your solution

```
class Barrel:
  def __init__(self, _capacity):
    self.capacity = _capacity

  def draw(self, bottle_capacity):
```

```
        # if the barrel has enough capacity, draw the entire bottle capacity and update t
        if self.capacity >= bottle_capacity:
            self.capacity -= bottle_capacity
            return bottle_capacity
        # if the barrel has less capacity than the bottle, draw the entire barrel capacit
        drawned = self.capacity
        self.capacity = 0
        return drawned
```

Test your solution

```
# Test case 1 (Example)
# We define a barrel containing 5 liters
b = Barrel(5)
# We fill a bottle containing 3 liters
b.draw(3)
# We try to fill another bottle containing 3 liters,
# but only 2 liters are left in the barrel
b.draw(3)
# We try to fill yet another bottle,
# but now barrel is empty
print(b.draw(3))
# Prints: 0

# Test case 2
a = Barrel(10)
b = a.draw(1)
print(b)

# Test case 3
a = Barrel(10)
```

```
b = a.draw(1)
b = a.draw(4)
print(b)

# Test case 4
a = Barrel(10)
b = a.draw(1)
b = a.draw(12)
print(b)
```

```
0
1
4
9
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 91: Car

Exercise Description

Define a class Car to simulate the behavior of a car. The user will be able to refuel the car, and then to drive, consuming fuel (if there is enough fuel). The class shall provide the following methods: Constructor: takes one float parameter fuel_consumption, expressed in liters per kilometer. The fuel tank is initially empty. refuel(quantity): when called, quantity liters of fuels are stored in the tank. drive(distance): ask to drive for distance kilometers, while consuming fuel. The method shall return the number of kilometers actually traveled: if there is not enough fuel, the car travels as much as possible, stopping as soon as tank becomes empty.

Your solution

```
class Car:
    def __init__(self, fuel_consumption):
        self.fuel_consumption = fuel_consumption
        self.tank = 0
    def refuel(self, quantity):
        self.tank += quantity
    def drive(self, distance):
        x = distance * self.fuel_consumption
        if x <= self.tank:
            self.tank -= x
            return distance
        else:
            d = self.tank / self.fuel_consumption
            self.tank = 0
            return d
```

Test your solution

```
# Test case 1 (Example)
c = Car(0.1)    # Our car consumes 0.1 liters per kilometer
# Let's refuel 20 liters
c.refuel(20)
# Try to drive for 150 km
traveled = c.drive(150)
print(int(traveled))        # Expected: 150 - ok

# Test case 2
c = Car(0.1)
c.refuel(20)
traveled = c.drive(150)
traveled = c.drive(100)
print(float(traveled))

# Test case 3
c = Car(0.1)
c.refuel(20)
traveled = c.drive(150)
traveled = c.drive(100)
traveled = c.drive(70)
print(float(traveled))

# Test case 4
c = Car(0.1)
c.refuel(20)
traveled = c.drive(150)
traveled = c.drive(100)
traveled = c.drive(70)
```

```python
c.refuel(30)
traveled = c.drive(70)
print(int(traveled))
```

```
150
50.0
0.0
70
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 [Visualize YOUR solution in Python Tutor]()

---

# Exercise 92: Cat

Exercise Description

Define a class Cat to represent cat characters in a videogame. A cat, as you know, has 9 lives. The class shall provide the following methods: Constructor: does not take any parameter. kill(): when called, the cat loses one of its lives is_alive(): returns True, if the cat is still alive (i.e., if it has lost at most 8 lives), False otherwise.

Your solution

```python
class Cat:
    def __init__(self):
        self.life = 9
    def kill(self):
        self.life -= 1
    def is_alive(self):
        return self.life > 0
```

Test your solution

```python
# Test case 1 (Example)
c = Cat()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
print(c.is_alive())

# Test case 2
c = Cat()
c.kill()
c.kill()
```

```python
c.kill()
c.kill()
c.kill()
c.kill()
print(c.is_alive())

# Test case 3
c = Cat()
print(c.is_alive())

# Test case 4
c = Cat()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
c.kill()
print(c.is_alive())
```

```
False
True
True
False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 93: Counter

Exercise Description

Define a class Counter to track frequencies. The class shall provide: Constructor: takes no parameters. increment(item): adds 1 to the count for given item. decrement(item): subtracts 1 from count for given item (count can't go below 0). get_most_frequent(): returns the item with highest count (if tie, return "tie")

Your solution

```
class Counter:
    def __init__(self):
        self.myItems = {}

    def increment(self, item):
        if item in self.myItems.keys():
            self.myItems[item] += 1
        else:
```

```
            self.myItems[item] = 1

    def decrement(self, item):
        if item in self.myItems.keys():
            self.myItems[item] -= 1

    def get_most_frequent(self):
        M = max(self.myItems.values())
        this = [(k, v) for (k, v) in self.myItems.items() if v == M]
        if len(this) > 1:
            return 'tie'
        else:
            this = this[0]
            return this[0]
```

Test your solution

```
# Test case 1 (Example)
c = Counter()
c.increment("a")
c.increment("b")
c.increment("a")
c.increment("a")
most_freq = c.get_most_frequent()
print(most_freq)

# Test case 2
c = Counter()
c.increment("x")
c.increment("x")
c.decrement("x")
```

```python
c.decrement("x")
c.decrement("x")
c.increment("y")
most_freq = c.get_most_frequent()
print(most_freq)

# Test case 3
c = Counter()
c.increment("dog")
c.increment("cat")
c.increment("dog")
c.increment("cat")
most_freq = c.get_most_frequent()
print(most_freq)

# Test case 4
c = Counter()
c.increment("python")
c.increment("python")
c.decrement("python")
most_freq = c.get_most_frequent()
print(most_freq)

# Test case 5
c = Counter()
c.increment("red")
c.increment("blue")
c.increment("red")
c.decrement("blue")
c.increment("green")
```

```
most_freq = c.get_most_frequent()
print(most_freq)
```

```
a
y
tie
python
red
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 94: Inventory

Exercise Description

Define a class Inventory to manage product stock. The class shall provide: Constructor: takes no parameters. add_product(product_id, quantity, price): adds new product with given quantity and unit price. sell(product_id, quantity): reduces product quantity if available.

Returns total price if successful, -1 if insufficient stock. get_most_valuable(): returns product_id of product with highest (quantity × price).

Your solution

```
class Inventory:
    def __init__(self):
        self.myItems = {}

    def add_product(self, product_id, quantity, price):
        self.myItems[product_id] = [quantity, price]

    def sell(self, product_id, quantity):
        if quantity <= self.myItems[product_id][0]:
            totalPrice = quantity * self.myItems[product_id][1]
            self.myItems[product_id][0] -= quantity
            return totalPrice
        else:
            return -1

    def get_most_valuable(self):
        this = {}
        for k, v in self.myItems.items():
            this[k] = v[0] * v[1]
        this = max(this.items(), key=lambda x: x[1])
        return this[0]
```

Test your solution

```python
# Test case 1 (Example)
i = Inventory()
i.add_product("A123", 5, 10.0)
i.add_product("B456", 3, 20.0)
i.add_product("C789", 10, 5.0)
most_valuable = i.get_most_valuable()
print(most_valuable)

# Test case 2
i = Inventory()
i.add_product("laptop", 10, 1000.0)
i.add_product("mouse", 20, 25.0)
sale_price = "{:.2f}".format(i.sell("laptop", 2))
print(sale_price)

# Test case 3
i = Inventory()
i.add_product("phone", 5, 500.0)
i.sell("phone", 3)
result = i.sell("phone", 3)
print(result)

# Test case 4
i = Inventory()
i.add_product("X001", 10, 50.0)
i.add_product("X002", 5, 200.0)
i.sell("X001", 8)
most_valuable = i.get_most_valuable()
print(most_valuable)

# Test case 5
```

```
i = Inventory()
i.add_product("table", 3, 100.0)
i.add_product("chair", 8, 50.0)
sale_price = "{:.2f}".format(i.sell("chair", 2))
i.add_product("desk", 4, 150.0)
print(sale_price)
```

```
B456
2000.00
-1
X002
100.00
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 95: Match

Exercise Description

Define a class Match to represent a soccer match. The class shall provide the following methods: Constructor: takes two string parameters (home_team, away_team) with the name of playing teams. goal(team): called when a team scores a goal. String team shall be the name of the team which scored the goal. get_score(): returns the current score, as a tuple with 4 elements (home_team, home_score, away_team, away_score). get_winning(): if a team is winning, returns the name of that team. Otherwise, returns "Tie".

Your solution

```
class Match:
    def __init__(self, home_team, away_team):
        self.home_team = home_team
        self.away_team = away_team
        self.home_score = 0
        self.away_score = 0

    def goal(self, team):
        if team == self.home_team:
            self.home_score += 1
        elif team == self.away_team:
            self.away_score += 1

    def get_score(self):
        return (self.home_team, self.home_score, self.away_team, self.away_score)

    def get_winning(self):
        if self.home_score == self.away_score:
            return "Tie"
        elif self.home_score > self.away_score:
```

```python
            return self.home_team
        elif self.home_score < self.away_score:
            return self.away_team
```

Test your solution

```python
# Test case 1 (Example)
m = Match("Liverpool", "Manchester")
m.goal("Manchester")
m.goal("Manchester")
m.goal("Liverpool")
w = m.get_winning()
print(w)

# Test case 2
m = Match("Liverpool", "Manchester")
m.goal("Manchester")
m.goal("Manchester")
m.goal("Liverpool")
s = m.get_score()
print(s[1])

# Test case 3
m = Match("Liverpool", "Manchester")
m.goal("Manchester")
m.goal("Manchester")
m.goal("Liverpool")
m.goal("Liverpool")
print(m.get_winning())

# Test case 4
```

```python
m = Match("Liverpool", "Manchester")
m.goal("Manchester")
m.goal("Manchester")
m.goal("Manchester")
m.goal("Manchester")
m.goal("Manchester")
m.goal("Manchester")
s = m.get_score()
print(s[3])
```

```
Manchester
1
Tie
6
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 96: Padlock

Exercise Description

Define a class Padlock, where each instance represents a combination padlock. A padlock has two states: unlocked and locked. A locked padlock can be unlocked only if you know the code The class shall provide the following methods: Constructor: takes a string parameter code, which is the numeric code that will unlock the padlock. Initially the padlock is unlocked is_locked(): shall return True if the padlock is currently locked, False otherwise lock(): shall lock the padlock unlock(code): called to try to unlock the padlock. If code is correct, the padlock shall become unlocked. If code is incorrect, the padlock state shall not change.

Your solution

```
class Padlock:
    def __init__(self, code):
        self.code = code
        self.state = 'unlocked'
    def is_locked(self):
        return self.state == 'locked'
    def lock(self):
        self.state = 'locked'
    def unlock(self, code):
        if code == self.code:
            self.state = 'unlocked'
        else:
            pass
```

Test your solution

```python
# Test case 1 (Example)
p = Padlock('1234')
print(p.is_locked())

# Test case 2
p = Padlock('1234')
p.lock()
print(p.is_locked())

# Test case 3
p = Padlock('1234')
p.lock()
p.unlock('0000')
print(p.is_locked())

# Test case 4
p = Padlock('1234')
p.lock()
p.unlock('0000')
p.unlock('1234')
p.unlock('9999')
print(p.is_locked())
```

```
False
True
True
False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 97: Parking Lot

Exercise Description

# ParkingLot

Define a class ParkingLot to manage parking spaces. The class shall provide: Constructor: takes number_of_spaces (int) as parameter. park_car(license_plate): assigns a space if available. Returns True if successful. remove_car(license_plate): removes car from lot. Returns True if car was present. get_usage_percent(): returns percentage of occupied spaces (0-100).

Your solution

```
class ParkingLot:
    def __init__(self, number_of_spaces):
        self.totalPlaces = number_of_spaces
        self.occupiedPlaces = 0
```

```
        self.licensePlates = set()

    def park_car(self, license_plate):
        if self.occupiedPlaces < self.totalPlaces:
            self.occupiedPlaces += 1
            self.licensePlates.add(license_plate)
            return True
        else:
            return False

    def remove_car(self, license_plate):
        if license_plate in self.licensePlates:
            self.occupiedPlaces -= 1
            self.licensePlates.remove(license_plate)
            return True
        else:
            return False

    def get_usage_percent(self):
        return 100 * self.occupiedPlaces / self.totalPlaces
```

Test your solution

```
# Test case 1 (Example)
p = ParkingLot(3)
p.park_car("ABC123")
p.park_car("DEF456")
usage = "{:.2f}".format(p.get_usage_percent())
print(usage)

# Test case 2
```

```python
p = ParkingLot(2)
p.park_car("CAR111")
p.park_car("CAR222")
result = p.park_car("CAR333")
print(result)

# Test case 3
p = ParkingLot(4)
p.park_car("X1")
p.park_car("X2")
p.remove_car("X1")
p.park_car("X3")
usage = "{:.2f}".format(p.get_usage_percent())
print(usage)

# Test case 4
p = ParkingLot(5)
p.park_car("AA11")
result = p.remove_car("BB22")
print(result)

# Test case 5
p = ParkingLot(10)
p.park_car("TEST1")
p.park_car("TEST2")
p.remove_car("TEST1")
p.remove_car("TEST2")
usage = "{:.2f}".format(p.get_usage_percent())
print(usage)
```

```
66.67
False
50.00
False
0.00
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 98: Personal Wallet

Exercise Description

Define a class Wallet to represent a personal digital wallet. The user will be able to load money in the wallet, and then to spend money to purchase items. The class shall provide the following methods: Constructor: does not take any parameter: it initialises an empty wallet (i.e., starting credit is 0) load(amount): when called, the amount of money amount shall be added to the wallet credit get_credit(): returns the current balance of the wallet purchase(price): purchases an item whose price is price. Returns True on success, i.e., if there is enough credit in the wallet the transaction is completed. Returns False if there is not enough credit, and the transaction is aborted (i.e., item is not purchased).

Your solution

```
class Wallet:
    def __init__(self):
        self.credit = 0

    def load(self, amount):
        self.credit = self.credit + amount

    def get_credit(self):
        return self.credit

    def purchase(self, price):
        if price <= self.credit:
            self.credit -= price
            return True
        else:
            return False
```

Test your solution

```
# Test case 1 (Example)
w = Wallet()
# Let's load some money
w.load(10.00)
# We buy an item
success = w.purchase(6.00)
```

```python
print(success)

# Test case 2
w = Wallet()
w.load(10.00)
success = w.purchase(6.00)
print(float(w.get_credit()))

# Test case 3
w = Wallet()
w.load(10.00)
success = w.purchase(6.00)
success = w.purchase(7.00)
print(success)

# Test case 4
w = Wallet()
w.load(10.00)
success = w.purchase(6.00)
success = w.purchase(7.00)
w.load(5.00)
success = w.purchase(7.00)
print(float(w.get_credit()))
```

```
True
4.0
False
2.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 99: Scale

Exercise Description

Define a class Scale to represent a traditional scale (balance) with two plates: a left and a right plate. The user will be able to incrementally load each plate, adding weight, and test whether the scale is in equilibrium or it tilts to the left or to the right.  The class shall provide the following methods:  Constructor: does not take any parameter: it initialises a scale with empty plates.  load(weight, plate): when called, the weight weight shall be added to plate plate. plate is a string that may be 'L' (left plate) or 'R' (right plate)  measure(): returns the current measurement outcome. The method returns a string that may be 'E' (equilibrium), 'L' (left plate is heavier) or 'R' (right plate is heavier).

Your solution

```
class Scale:
    def __init__(self):
        self.left = 0
```

```
        self.right = 0
    def load(self,weight,plate):
        if plate == 'R':
            self.right += weight
        else :
            self.left += weight
    def measure(self):
        if self.left > self.right:
            return 'L'
        elif self.left < self.right:
            return 'R'
        else:
            return 'E'
```

Test your solution

```
# Test case 1 (Example)
s = Scale()
m = s.measure()
print(m)

# Test case 2
s = Scale()
m = s.measure()
s.load(2.5, 'L')
s.load(4, 'R')
m = s.measure()
print(m)

# Test case 3
s = Scale()
```

```python
m = s.measure()
s.load(2.5, 'L')
s.load(4, 'R')
m = s.measure()
s.load(3, 'L')
m = s.measure()
print(m)

# Test case 4
s = Scale()
m = s.measure()
s.load(2.5, 'L')
s.load(4, 'R')
m = s.measure()
s.load(3, 'L')
m = s.measure()
s.load(1.5, 'R')
m = s.measure()
print(m)
```

```
E
R
L
E
```

### Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

[🔗 Visualize YOUR solution in Python Tutor]

---

# Exercise 100: Student

Exercise Description

Define a class Student to track student grades. The class shall provide: Constructor: takes name (string) as parameter. add_grade(grade): adds a new grade (float) to student's grades. get_average(): returns the average of all grades. is_passing(): returns True if average is >= 60, False otherwise.

Your solution

```
class Student:
    def __init__(self, name):
        self.studentName = name
        self.studentGrades = []

    def add_grade(self, grade):
        self.studentGrades.append(grade)

    def get_average(self):
        return sum(self.studentGrades) / len(self.studentGrades)

    def is_passing(self):
```

```
        avg = self.get_average()
        if avg >= 60:
            return True
        else:
            return False
```

Test your solution

```
# Test case 1 (Example)
s = Student("Alice")
s.add_grade(45.5)
s.add_grade(55.5)
result = s.is_passing()
print(result)

# Test case 2
s = Student("John")
s.add_grade(85.5)
s.add_grade(92.0)
s.add_grade(88.5)
average = "{:.2f}".format(s.get_average())
print(average)

# Test case 3
s = Student("Bob")
s.add_grade(58.0)
s.add_grade(62.0)
average = "{:.2f}".format(s.get_average())
print(average)

# Test case 4
```

```python
s = Student("Carol")
s.add_grade(75.5)
average = "{:.2f}".format(s.get_average())
print(average)

# Test case 5
s = Student("David")
s.add_grade(82.25)
s.add_grade(89.75)
s.add_grade(95.50)
s.add_grade(91.25)
average = "{:.2f}".format(s.get_average())
print(average)
```

```
False
88.67
60.00
75.50
89.69
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```python
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

# Exercise 101: Telephone Account

Exercise Description

## TelephoneAccount

Define a class TelephoneAccount to keep track of the phone calls of a subscriber of a telephone line. The class shall provide the following methods. Constructor: takes two float parameters (call_setup_fee, per_minute_fee). add_call(duration): called when a phone call is performed. If duration is 0, the call is unsuccessful and nothing is charged. Otherwise, duration is the length of the call, in minutes. The subscriber is charged the call setup fee, plus a variable cost, depending on the call duration. get_bill(): returns the total price to be paid for all the calls that have been performed so far.

Your solution

```
class TelephoneAccount:
    def __init__(self, call_setup_fee, per_minute_fee):
        self.call_setup_fee = call_setup_fee
        self.per_minute_fee = per_minute_fee
        self.amount = 0

    def add_call(self, duration):
        if duration == 0:
            pass
        else:
```

```
            self.amount += self.call_setup_fee + self.per_minute_fee * duration

    def get_bill(self):
        return self.amount
```

Test your solution

```
# Test case 1 (Example)
a = TelephoneAccount(0.1, 0.2)
a.add_call(2)
a.add_call(0)
a.add_call(1.5)
b = a.get_bill()
print(float(b))

# Test case 2
a = TelephoneAccount(0.1, 0.2)
a.add_call(2)
a.add_call(0)
a.add_call(1)
b = a.get_bill()
a.add_call(1)
print(float(a.get_bill()))

# Test case 3
a = TelephoneAccount(1, 3)
a.add_call(2)
a.add_call(0)
a.add_call(1)
b = a.get_bill()
a.add_call(1)
```

```
print(float(a.get_bill()))

# Test case 4
a = TelephoneAccount(1, 3)
a.add_call(2)
a.add_call(0)
a.add_call(0)
b = a.get_bill()
a.add_call(1)
print(float(a.get_bill()))
```

```
0.9
1.1
15.0
11.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

🔗 Visualize YOUR solution in Python Tutor

---

# Exercise 102: Temperature

Exercise Description

Define a class Temperature to handle temperature conversions. The class shall provide: Constructor: takes temperature value (float) and unit ('C' or 'F' as string). to_celsius(): returns temperature converted to Celsius if in Fahrenheit, or same value if already Celsius. to_fahrenheit(): returns temperature converted to Fahrenheit if in Celsius, or same value if already Fahrenheit. is_freezing(): returns True if temperature is at or below freezing point (0°C/32°F), False otherwise. Please note that: C = (5/9) * (F - 32) and F = (C * 9/5) + 32

Your solution

```
class Temperature:
    def __init__(self, value, unit):
        self.value = value
        self.unit = unit

    def to_celsius(self):
        if self.unit == 'F':
            return (5/9) * (self.value - 32)
        else:
            return self.value

    def to_fahrenheit(self):
        if self.unit == 'C':
            return (self.value * 9/5) + 32
        else:
            return self.value

    def is_freezing(self):
```

```
        if self.unit == 'C' and self.value <= 0:
            return True
        if self.unit == 'F' and self.value <= 32:
            return True
        return False
```

Test your solution

```
# Test case 1 (Example)
t = Temperature(0.0, 'C')
result = t.is_freezing()
print(result)

# Test case 2
t = Temperature(98.6, 'F')
celsius = "{:.2f}".format(t.to_celsius())
print(celsius)

# Test case 3
t = Temperature(25.0, 'C')
fahrenheit = "{:.2f}".format(t.to_fahrenheit())
```